

**INSTITUTO SUPERIOR TECNOLÓGICO
VIDA NUEVA
SEDE MATRIZ**



CARRERA:

TECNOLOGÍA SUPERIOR EN DESARROLLO DE SOFTWARE

ASIGNATURA: ANALISIS DE DISEÑO DE SOFTWARE

CÓDIGO: 24241041

MODALIDAD: PRESENCIAL

JORNADA: MATUTINA

NIVEL: CUARTO

**TEMA:
PORTAFOLIO GENERAL**

**AUTOR:
CISNEROS MONTOYA MATEO ESTEBAN**

DOCENTE: ING. GAVILANES OBREGON KEVIN ALEXANDER.MG.

MARZO 2025 – AGOSTO 2026

QUITO – ECUADOR



MISION Y VISION

INSTITUCIONAL

El ITUVN es una institución que forma profesionales líderes con alto valor humano, vocación de servicio, practica de principios éticos, espirituales y comprometidos con el desarrollo social, productivo y sustentable, donde la tecnología es la herramienta fundamental para satisfacer los requerimientos actuales de la educación, basada en los estándares de calidad que búsqueda soluciones críticas, creativas y sustentables en la intervención ante los problemas actuales

VISIÓN

El ITUVN se consolidará como una institución pionera en la educación técnica y tecnológica formando profesionales, a través de una educación altamente competitiva en el campo laboral; con excelentes servicios y procesos permanentes de evaluación integral, apoyados en los principios y valores institucionales, enfocados en la investigación e innovación tecnológica y la contribución al desarrollo sustentable del país.



PLANIFICACION



Tecnológico Universitario
Vida Nueva

CALENDARIO ACADÉMICO

OCTUBRE 2024 - MARZO 2025
MODALIDAD PRESENCIAL - HÍBRIDA

SEDE MATRIZ
(Guamaní)

FECHA	ACTIVIDAD
30/09/2024 - 04/10/2024	Jornada de planificación microcurricular.
07/10/2024 - 11/10/2024	Capacitación docente.
12/10/2024	Evaluación diagnóstica de ingreso.
14/10/2024	Inicio del periodo académico octubre 2024-marzo 2025.
14/10/2024 - 05/11/2024	Proceso de selección de ayudantes de docencia.
08/11/2024	Jornada de crecimiento personal.
14/11/2024	Jornada de Actualización Científica y Profesional TUVN Innova Talks.
05/12/2024	Programa por la Fundación de Quito.
16/01/2025 - 17/01/2024	Feria de Tecnología y Orientación Profesional.
27/03/2025	Encuentro Interinstitucional de Cooperación y Desarrollo 2025.
18/03/2025	Finalización del periodo académico octubre 2024-marzo 2025.

DÍAS LIBRES POR FERIADO O EVENTOS INSTITUCIONALES

01,02 Y 03/11/2024	Día de los difuntos.
04/11/2024	Independencia de Cuenca.
08/11/2024	Jornada de Crecimiento Personal.
14/11/2024	Jornada de Actualización Científica y Profesional TUVN Innova Talks.
05/12/2024	Programa por la Fundación de Quito.
06/12/2024	Fundación de Quito.
23, 24 y 25/12/2024	Navidad.
30 y 31/12/2024	Fin de año.
01/01/2025	Año nuevo.
16 y 17/01/2025	Feria de Tecnología y Orientación Profesional.
03 y 04 /03/2025	Carnaval.
27/03/2025	Encuentro Interinstitucional de Cooperación y Desarrollo 2025.



HORARIO DE CLASES

NIVEL	F.INICIO	CODIGO	ASIGNATURA	CRÉDITOS	HORARIO	AULA/TALLER
CUARTO	2026-06-16	26261041	ANALISIS Y DISEÑO DE SOFTWARE	1	08:00-10:00	F6
CUARTO	2026-04-15	26261031	BASE DE DATOS II	2	08:00-10:00	LABH4
CUARTO	2026-08-25	26261001	LEGISLACION LABORAL	1	08:00-10:00	F6
CUARTO	2026-07-15	26261061	PROGRAMACION MOVIL	2	10:00-12:00	LAB1
CUARTO	2026-04-15	26261051	PROGRAMACION WEB	4	10:00-12:00	LAB1





HORARIO DE ACCION TUTORIAL

HORARIO DE ACCIÓN TUTORIAL			
CARRERA	MODALIDAD PRESENCIAL / HÍBRIDA JORNADA MATUTINA / NOCTURNA	JUEVES	SÁBADO
MECÁNICA AUTOMOTRIZ / ELECTROMECÁNICA AUTOMOTRIZ	12:30-13:30 16:50-17:50	Mg. Edwin Chilibunga Mg. Jorge Flores Mg. Juan Jara Mg. Luis Ica Mg. Cristian Guachaman Mg. Denis Ugoño Mg. Edison Córdova	MODALIDAD SEMIPRESENCIAL Mecánica Automotriz <i>Asignaturas del horario 08 - 12</i> 07:00-07:50 <i>Asignaturas del horario 13 - 18</i> 18:10-19:00
MECÁNICA INDUSTRIAL	12:30-13:30 16:50-17:50	Mg. Raúl Cusi Mg. Víctor López Mg. Placido Cruz Mg. Pamela Villareal Mg. Rodrigo Díaz	
ELECTROMECÁNICA AUTOMATIZACIÓN E INSTRUMENTACIÓN	12:30-13:30 16:50-17:50	Mg. Carlos Ruiz Mg. Byron Mollay Mg. Edwin Mollay	
DESARROLLO DE SOFTWARE	12:30-13:30 16:50-17:50	Mg. Daniel Astor Linda Fabian Jaramila Ing. Jorge Ballester Ing. Luis Villanueva Mg. Daniel Camacho	MODALIDAD EN LÍNEA Administración Desarrollo de Software Marketing Digital y Comercio Electrónico <i>Asignaturas del horario 08 - 12</i> 07:00-07:50 <i>Nota: no hay acción tutorial el día en que inicia una asignatura</i> <i>Asignaturas del horario 13 - 17</i> 17:10-18:00 <i>Nota: no hay acción tutorial el día en que inicia una asignatura y, también, evaluación final</i>
CONTABILIDAD	12:30-13:30 16:50-17:50	Ing. Tereza Alvarado Ing. Jessica Góngora	
ADMINISTRACIÓN	12:30-13:30 16:50-17:50	Mg. Estelita Segura Mg. Paula Flores Edu. Carla Maldonado Mg. Jessica Guipé Mg. Cesar Tipón Linda Verónica Hernández Ing. Juliana Jiménez	
EDUCACIÓN BÁSICA / DOCENCIA E INNOVACIÓN EDUCATIVA	12:30-13:30 16:50-17:50	Mg. Daniel Rincón Mg. Mariana Quispe Mg. Natalia Contreras Mg. Diana Tardón Mg. Carolina Riverales Mg. Carlo Quispe Linda Elizabeth Arce	
TURISMO	12:30-13:30 16:50-17:50	Mg. Geovanny Quispe Mg. Estelita Segura	
ENFERMERÍA	12:30-13:30 16:50-17:50	Doc. Edgar Muñoz Linda Carla Blecita Linda Jocelyn Páez Obed. Carla Balanconde Linda Griselda Cipriano Linda Jenny Colipa Linda Rosalva Ordóñez Linda Brandon Nide Linda Rocio Samaniego	
CENTRO DE IDIOMAS	16:50-17:50	Linda Ricardo Gálvez Linda Katherine Sanguinera Linda Jorge Guzmán Linda Jessica Tizadillo Linda María Ríos	



SYLLABUS



PROGRAMA DE ESTUDIO DE LA ASIGNATURA

(Versión 4)

1. DATOS INFORMATIVOS

1.1.	Carrera:	Tecnología Superior en Desarrollo de Software			
1.2.	Modalidad:	Presencial			
1.3.	Nombre de la asignatura:	Análisis y diseño de software		DSUP13	
1.4.	Nivel:	Cuarto			
1.5.	Unidad de organización curricular:	Profesional			
1.6.	Número total de horas de la asignatura / créditos:	48 horas / 1 créditos			
1.7.	Número de horas del componente de aprendizaje en contacto con el docente:			16	
1.8.	Número de horas del componente de aprendizaje práctico-experimental:			16	
1.9.	Número de horas del componente de aprendizaje autónomo:			16	
1.10.	Prerrequisito:	Asignatura:	Ninguna	Código:	Ninguno
1.11.	Correquisito:	Asignatura:	Ninguna	Código:	Ninguno
1.12.	Línea(s) de vinculación con la sociedad y servicio a la comunidad relacionadas:		Aplicación de conocimientos prácticos en la solución de hardware y software. Aplicación de conocimiento en el desarrollo y publicación de páginas web. Software, desarrollo y análisis de aplicativos.		
1.13.	Línea(s) de investigación relacionadas:		Sistemas de información para la gestión e innovación social.		

2. CARACTERIZACIÓN DE LA ASIGNATURA

2.1. OBJETIVO DE LA ASIGNATURA

Aplicar patrones de diseño modernos utilizando UML para la construcción de software, aprovechando herramientas que automaticen la generación de código a partir de los diagramas. Implementar progresivamente estos patrones en proyectos de software, con un enfoque en la refactorización y evaluación continua de la calidad del código. Fomentar una valoración ética en el diseño de software, incentivando el trabajo colaborativo y la retroalimentación constante entre pares.

2.2. RESULTADOS DE APRENDIZAJE DEL PERFIL DE EGRESO A LOS QUE CONTRIBUYE LA ASIGNATURA

- Maneja las técnicas y el modelado UML previo al desarrollo de software basado en parámetros de calidad.
- Elabora diagramas UML en base a los requerimientos de usuario aplicando normas de procesos y calidad de software.
- Efectúa el análisis de los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software Orientado a Objetos.

2.3. CONTRIBUCIÓN A LAS PRÁCTICAS PREPROFESIONALES

En la ejecución de las prácticas preprofesionales el estudiante:

- Aplicará las herramientas de diseño UML para representar la construcción de sistemas informáticos mediante esquemas gráficos.
- Simplificará el diseño de la construcción de un sistema informático haciendo uso de los diferentes tipos de diagramas.
- Dominará las reglas y normas en las construcciones específicas de diagramas UML.
- Comprenderá cómo representar un software de manera gráfica usando UML para resolver varios problemas del modelado de software.

2.4. CONTRIBUCIÓN A LAS CAPACIDADES GENERALES PARA EL APRENDIZAJE Y FORMACIÓN CULTURAL

En cuanto a las capacidades generales para el aprendizaje y formación cultural, el estudiante será capaz de:

- Promover acciones enfocadas en la inclusión de género, etnia y capacidades diversas, principalmente la interculturalidad desde la óptica científica que concibe al desarrollo de la ciencia desde la historia de los intercambios científicos de las diferentes culturas.
 - Identificar vulnerabilidades de seguridad en los programas y aplicaciones. para proteger la información personal y financiera de las personas, y para garantizar la seguridad informática social.
 - Aplicar nuevas tecnologías y soluciones para diferentes problemas, lo que puede impulsar la innovación en la sociedad y mejorar la calidad de vida de las personas.
-



3. UNIDADES DIDÁCTICAS

NÚMERO DE LA UNIDAD:	Uno		
NOMBRE DE LA UNIDAD:	Introducción a UML.		
OBJETIVO DE LA UNIDAD:	Aplicar técnicas y modelado UML antes del desarrollo de software basado en parámetros de calidad para mejorar la comprensión de requisitos y objetivos, fortalecer la precisión en el diseño, y promover actitudes disciplinadas y colaborativas que aseguren la entrega de resultados de alta calidad.		
CONTENIDO MÍNIMO	ORGANIZACIÓN DEL APRENDIZAJE		
	APRENDIZAJE EN CONTACTO CON EL DOCENTE	APRENDIZAJE PRÁCTICO EXPERIMENTAL	APRENDIZAJE AUTÓNOMO
	Horas	Horas	Horas
1. Fundamentos de análisis y diseño de sistemas 2. Proceso de análisis de sistemas de información 2.1 Requerimientos: modelamiento funcional y estructural 2.2 Fuentes de datos: sistema actual, internas y externas 3. Recopilación de información 4. Especificación de requisitos de software (IEEE-830) 5. Fundamentos de UML 6. Análisis y diseño orientado a objetos 6.1 Casos de uso 6.2 Diagrama de clases y asociaciones: generalización, agregación, composición 7. Metodologías ágiles en desarrollo de software 7.1 Principios ágiles 7.2 SCRUM: roles, eventos y artefactos 7.3 Integración de SCRUM con análisis y diseño	8	2	3
ACTIVIDADES DE APRENDIZAJE	Actividades de adquisición: Clases de UML (Lenguaje Unificado de Modelado) Clases de elementos generales que utiliza UML para los diferentes	Actividades prácticas: Realizar un levantamiento de procesos para el sistema de Tickets de soporte técnico en Visio / herramientas BPMM Nombre de la guía:	Actividades de investigación: Consulta bibliográfica respecto a las técnicas de levantamiento de requerimientos de software, realizar una tabla comparativa con los



	diagramas en el modelamiento de software. Actividades de colaboración: Simular el proceso de ingeniería de requisitos para un proyecto de software (identificar, documentar y clasificar los requisitos funcionales y no funcionales). Modelar un caso de uso de una aplicación simple utilizando UML y organizar las tareas en un sprint ágil, asignando prioridades y responsables. Desarrollar un organizador gráfico sobre los componentes de cada diagrama (clases, caso de usos, secuencia) en la herramienta en línea mindomo, generando la información necesaria en la herramienta de IA ChatGPT	Desarrollo de la guía de práctica "UML" Objetivo: Reconocer los requerimientos de software previo a la diagramación UML de un sistema académico. Sustento: La práctica propuesta posibilita que el estudiante comprenda el sistema desde la perspectiva del usuario final. para centrarse en las interacciones entre el usuario y el sistema.	conceptos generados por la IA de ChatGPT Investigación dando uso a ChatGPT respecto a las metodologías, técnicas y herramientas para modelar el proceso de software. Consulta bibliográfica sobre las técnicas de recolección, análisis y documentación de requisitos en ingeniería de software, evaluando su impacto en la calidad del sistema y la satisfacción del usuario final. Actividades de producción: Elaboración del informe de la guía práctica "UML".
RECURSOS DIDÁCTICOS (Ver anexo)	Proyector Parlantes Pc Libros Diapositivas Videos SOFTWARES Microsoft Visio Modelio StarUML	Guía de Practica Libros Textos Diapositivas SOFTWARES Enterprise architect Microsoft Visio Modelio StarUML	Campus Virtual YouTube Repositorios Correo Institucional SOFTWARES Microsoft Visio Bizagi Modelio StarUML
ESCENARIOS DE APRENDIZAJE (Ver anexo)	Aula Laboratorio de computo	Laboratorio de computo	
CRITERIOS DE EVALUACIÓN (Ver anexo)	Criterio: Reconoce los diferentes tipos de diagramas UML y su utilidad (Prueba Escrita)	Criterio: Aplica los diagramas de casos de uso dentro de la documentación del software (Lista de Cotejo)	Criterio: Crea de manera eficiente diagramas estáticos y dinámicos (Prueba Oral)

NÚMERO DE LA UNIDAD:	Dos
-----------------------------	-----



NOMBRE DE LA UNIDAD:	Tipos de Diagramas.		
OBJETIVO DE LA UNIDAD:	Elaborar diagramas UML basados en los requerimientos del usuario y aplicando normas de procesos y calidad de software implica primero comprender profundamente las necesidades del usuario mediante un análisis detallado y una comunicación efectiva.		
CONTENIDO MÍNIMO	ORGANIZACIÓN DEL APRENDIZAJE		
	APRENDIZAJE EN CONTACTO CON EL DOCENTE	APRENDIZAJE PRÁCTICO EXPERIMENTAL	APRENDIZAJE AUTÓNOMO
1. Diagramas UML 1.1 Diagramas de secuencia 1.2 Diagramas de colaboración 1.2.1 Diagrama de estado 1.3 Diagrama de actividad 1.4 Diagramas de componentes 1.5 Diagramas de implementación 1.6 Diagramas de relación de entidad	Horas	Horas	Horas
	6	1	3
ACTIVIDADES DE APRENDIZAJE	Actividades de adquisición: Clases de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) en la herramienta CASE Power Designer.	Actividades prácticas: Nombre de la guía: Desarrollo de la guía de práctica "Casos de Usos" Objetivo: Identificar los diagramas de caso de uso, sus componentes y la importancia que tiene dentro de un proyecto de software. Sustento: La práctica propuesta posibilita que el estudiante proporcione una base para la prueba y validación del sistema permitiendo definir el comportamiento del sistema	Actividades de discusión: Observar el video tutorial relacionado al Diagrama de Caso de Usos Actividades de investigación: Investigar la creación de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) en publicaciones y/o buscadores académicos Link: - Google Scholar: https://scholar.google.com/ - Dialnet: https://dialnet.unirioja.es/ - Scielo: https://scielo.org/es/ Actividades de producción: Elaboración del informe de la guía práctica "Casos de Usos".



RECURSOS DIDÁCTICOS (Ver anexo)	Proyector Parlantes Pc Libros Diapositivas Videos SOFTWARES Microsoft Visio Modelio StarUML	Guia de Practica Libros Textos Diapositivas SOFTWARES Enterprise architect Microsoft Visio Modelio StarUML	Campus Virtual YouTube Repositorios Correo Institucional Libros Fotocopias SOFTWARES Microsoft Visio Bizagi Modelio StarUML
ESCENARIOS DE APRENDIZAJE (Ver anexo)	Aula Laboratorio de computo	Laboratorio de computo	
CRITERIOS DE EVALUACIÓN (Ver anexo)	Criterio: Modelar distintos aspectos de sistemas de software de manera eficiente. (Lista de Cotejo)	Criterio: Crea diagramas de caso de uso, sus diferentes componentes (Prueba Oral)	Criterio: Aplica de manera correcta los atributos dentro de los diagramas correspondientes (Prueba Escrita)



NÚMERO DE LA UNIDAD:	Tres		
NOMBRE DE LA UNIDAD:	Herramientas CASE para el modelado		
OBJETIVO DE LA UNIDAD:	Cumplir con los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software orientado a objetos implica comprender profundamente las necesidades del usuario, aplicando un proceso estructurado para identificar funcionalidades y relaciones.		
CONTENIDO MÍNIMO	ORGANIZACIÓN DEL APRENDIZAJE		
	APRENDIZAJE EN CONTACTO CON EL DOCENTE	APRENDIZAJE PRÁCTICO EXPERIMENTAL	APRENDIZAJE AUTÓNOMO
1. Vistas UML	Horas	Horas	Horas
1.1 Archivos de proyecto			
1.2 Estructura de las ventanas			
1.3 Vistas de la arquitectura			
1.4 Vista de casos de uso, use case view			
1.5 Vista lógica, logical view			
1.6 Vista de componentes, component view			
1.7 Vista de despliegue, deployment view			
1.8 Vista de diseño de la arquitectura			
1.9 Modelado de casos de uso			
1.10 Creación de casos de uso			
1.11 Asociación			
1.12 Diagrama de casos de uso			
1.13 Relaciones entre casos de uso			
2. Exportación: public, private, protected	13	2	10
2.1 Atributos y sus clases			
2.2 Modelado del análisis			
2.3 Modelo de clases			
3. Agregación y asociación			
3.1 Definir una clase asociación			
3.2 Crear una clase parametrizada			
3.3 Crear diagramas de colaboración			
3.3.1.1 Generar automáticamente código Java a partir del modelo UML			
3.3.1.2 Generación de informes.			



ACTIVIDADES DE APRENDIZAJE	Actividades de adquisición: Clases de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) Clases de la exportación: public, private, protected. Clases sobre los tipos de UML; Relaciones, Composición, Agregación, Asociación, Dependencia, Generalización, Realización. Aplicación de la evaluación final de la asignatura. Socialización de las notas parciales de la asignatura juntamente con la retroalimentación de su contenido. Actividades de colaboración: Desarrollar un mapa mental sobre las metodologías de desarrollo y los ciclos de vida del software, en la herramienta en línea mindomo Link: https://www.mindomo.com/es , generar una diapositiva sobre este tema aplicando la IA de tome en www.tomeapp	Actividades prácticas: Desarrollo de la guía de práctica "Diagramas de Actividades" Nombre de la guía: "Diagramas de Actividades" Objetivo: Construir el diagrama de objetos previo a un análisis de funcionalidades, para un sistema académico. Sustento: La práctica propuesta posibilita que el estudiante identificar los pasos necesarios para completar un proceso y asegurarse de que se hayan incluido el diseño de manera exitosa	Actividades de discusión: Aportar en un foro dentro de la plataforma sobre los paradigmas de programación y el análisis y diseño de software, generando información en la IA ChatGPT, haciendo un análisis de estos resultados. Actividades de investigación: Investigación de aspectos relacionados a la temática del trabajo final de asignatura Actividades de producción: Elaboración del informe del proyecto final de asignatura
RECURSOS DIDÁCTICOS (Ver anexo)	Proyector Parlantes Pc Libros Diapositivas Videos SOFTWARES Microsoft Visio Modelio StarUML	Guía de Practica Libros Textos Diapositivas SOFTWARES Enterprise architect Microsoft Visio Modelio StarUML	Campus Virtual YouTube Repositorios Correo Institucional Libros Fotocopias SOFTWARES Microsoft Visio Bizagi Modelio StarUML



ESCENARIOS DE APRENDIZAJE (Ver anexo)	Aula Laboratorio de computo	Laboratorio de computo	
CRITERIOS DE EVALUACIÓN (Ver anexo)	Criterio: Aplicar herramientas CASE para el modelado de software (Lista de Cotejo)	Criterio: Documenta de manera eficiente los procesos de un software (Prueba Oral)	Criterio: Genera prototipos diagramas y vistas de un sistema y sus requerimientos (Prueba Escrita)

4. METODOLOGÍAS DE ENSEÑANZA

METODOLOGÍA	MÉTODO	ESTRATÉGICA
	Controlado por el docente	Clase expositiva o lección magistral Deductivo Enunciación Comprobación Aplicación Analogico-comparativo Observación Comparación Conclusiones

	Controlado por el estudiante	Método activo o clases prácticas Enunciación Aclaración Resolución Comisión Planteamiento del problema Análisis Conclusiones
	Controlado por el docente y el estudiante	Aprendizaje basado en proyectos Diseño Planificación Ejecución Evaluación Observación Construcción de supuestos preliminares Búsqueda de información Establecimiento de semejanzas y diferencias Elaboración de descripciones sobre lo observado Taller Resolución de problemas

SUSTENTO:

Controlado por el docente

- Clase expositiva o lección magistral
La estrategia organiza conceptos complejos en módulos claros, facilitando su comprensión a través de un enfoque sistemático y ejemplos prácticos. Este método profundiza en los temas, enriqueciendo el aprendizaje y preparando a los estudiantes de manera efectiva para enfrentar escenarios fiscales reales.
- Deductivo
El método deductivo, dirigido por el docente, organiza la enseñanza de manera sistemática, estableciendo objetivos y actividades claras. El docente guía el proceso educativo, proporcionando información directa para asegurar la comprensión de conceptos fundamentales antes de aplicarlos a situaciones complejas. Este enfoque es ideal para materias que requieren precisión y una sólida base teórica antes de la práctica. El papel del docente es crucial para estructurar el aprendizaje y asegurar que los estudiantes adquieran conocimientos esenciales..

Controlado por el estudiante

- Método activo o clases prácticas
Promueve la participación activa de los estudiantes, facilitando una comprensión más profunda y la retención del contenido teórico mediante la aplicación práctica y la resolución de problemas en escenarios reales. Este enfoque permite a los estudiantes conectar teoría y práctica, desarrollar habilidades críticas y reforzar su capacidad para enfrentar situaciones complejas de manera autónoma y eficaz..
- Comisión

Permite a los estudiantes analizar situaciones reales, desarrollar habilidades críticas y aplicar conceptos teóricos en un contexto práctico, promoviendo así un aprendizaje profundo y contextualizado

Controlado por el docente y el estudiante

- Aprendizaje basado en proyectos
Facilita la enseñanza del modelado de software al permitir a los estudiantes aplicar conceptos teóricos a problemas reales, promover el trabajo colaborativo y desarrollar habilidades prácticas esenciales.
- Observación
Permite a estudiantes y docentes evaluar fenómenos directamente, promoviendo una comprensión empírica y detallada. Además, fomenta el desarrollo de habilidades analíticas y críticas, esenciales para interpretar datos con precisión en situaciones reales.
- Taller
Involucra activamente a estudiantes y docentes en actividades prácticas colaborativas, facilitando el aprendizaje del modelado de software mediante la experiencia directa y el intercambio de ideas, lo que enriquece la comprensión y aplicación de los conceptos teóricos.

5. BIBLIOGRAFÍA

BIBLIOGRAFÍA BÁSICA:	
1	Booch, G., Rumbaugh, J., Jacobson, I. (2006). <i>El Lenguaje Unificado de Modelado Uml 2.0</i> (2ª ed.). Pearson. (007.4035 - Libro Físico)
2	Debrauwer, L., Evain, Y., Ferrandez, S. (2021). <i>Design Patterns en Php. los 23 Patrones de Diseño: Descripciones y Soluciones Ilustradas en Uml 2 y Php</i> (2ª ed.). Eni. (005.8069 - Libro Físico)
SUSTENTO:	Científico – técnico Los libros son una referencia valiosa en el campo del desarrollo de software, ya que ofrece un enfoque técnico exhaustivo sobre los patrones de diseño. Presenta descripciones detalladas y soluciones ilustradas en UML 2 y PHP, lo que facilita una comprensión profunda y la aplicación práctica de conceptos avanzados. Su enfoque detallado en la implementación y ejemplos claros ayuda tanto a estudiantes como a profesionales a dominar patrones de diseño y mejorar la calidad del software. Didáctico Los libros ofrecen un sólido sustento didáctico al presentar patrones de diseño con descripciones detalladas y soluciones ilustradas en UML 2 y PHP. Este enfoque facilita la comprensión de conceptos complejos a través de ejemplos prácticos y aplicables, promoviendo un aprendizaje activo y aplicado. Además, la estructura del libro, que combina teoría con ejercicios prácticos, permite a los estudiantes y profesionales desarrollar habilidades técnicas mientras refuerzan su conocimiento mediante la resolución de problemas reales. Esta metodología didáctica asegura una integración efectiva de los patrones de diseño en el proceso educativo y profesional.
BIBLIOGRAFÍA DE CONSULTA:	
1	Alarcón, R. (2000). <i>Diseño Orientado a Objetos con Uml</i> . Grupo Eidos Consultoría y Documentación Informática. (007.55 - Libro Digital)



6. SISTEMA DE EVALUACIÓN GENERAL

OBJETIVO DE LA UNIDAD	CRITERIO DE EVALUACIÓN	ACTIVIDAD	PRODUCTO
Aplicar técnicas y modelado UML antes del desarrollo de software basado en parámetros de calidad para mejorar la comprensión de requisitos y objetivos, fortalecer la precisión en el diseño, y promover actitudes disciplinadas y colaborativas que aseguren la entrega de resultados de alta calidad.	Reconoce los diferentes tipos de diagramas UML y su utilidad	Reconocer las diferentes ilustraciones con su símbolo en UML	Prueba Escrita
	Aplica los diagramas de casos de uso dentro de la documentación del software	Documenta el software de manera correcta bajo la norma ISO respectiva	Lista de Cotejo
	Crea de manera eficiente diagramas estáticos y dinámicos	Demuestra sus conocimientos sobre los diferentes diagramas	Prueba Oral
Elaborar diagramas UML basados en los requerimientos del usuario y aplicando normas de procesos y calidad de software implica primero comprender profundamente las necesidades del usuario mediante un análisis detallado y una comunicación efectiva.	Modelar distintos aspectos de sistemas de software de manera eficiente.	Modela a través del software los tipos de sistemas con estándar UML	Lista de Cotejo
	Crea diagramas de caso de uso, sus diferentes componentes	Enunciar los diferentes tipos de diagramas que tiene UML	Prueba Oral
	Aplica de manera correcta los atributos dentro de los diagramas correspondientes	Diagrama de manera lógica y ordenada los diagramas solicitados de manera correcta	Prueba Escrita
Cumplir con los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software orientado a objetos implica comprender profundamente las necesidades del usuario, aplicando un proceso	Aplicar herramientas CASE para el modelado de software	Genera diagramas a través de las herramientas CASE de modelado	Lista de Cotejo
	Documenta de manera eficiente los procesos de un software	Genera los diagramas de procesos de los requisitos de software en UML	Prueba Oral



estructurado para identificar funcionalidades y relaciones.			
	Genera prototipos diagramas y vistas de un sistema y sus requerimientos	Demuestra la implementación de las vistas de un sistema	Prueba Escrita



7. REVISIÓN Y APROBACIÓN

ELABORADO	REVISADO	APROBADO
COMISIÓN DE LA CARRERA FECHA: 19/09/2025 <i>Coordinador de carrera Sede Matriz:</i> <i>Mg. Darwin Arias</i> <i>Coordinador de carrera Campus Norte:</i> <i>Ing. Kevin Toapanta</i> <i>Docentes de la carrera</i>	COORDINACIÓN ACADÉMICA FECHA: 24/09/2025 <i>Coordinador Académico Sede Matriz</i> <i>Coordinador Académico Campus Norte</i> <i>Coordinador Académico En línea</i>	VICERRECTORADO RECTORADO FECHA: 30/09/2025 <i>Vicerrectora General</i> <i>Rector</i>

8. HISTORIAL DE ACTUALIZACIONES

VERSIÓN	FECHA DE REVISIÓN	FECHA DE ACTUALIZACIÓN	ACTUALIZACIÓN	SUSTENTO DE LA ACTUALIZACIÓN
Versión 1	-----	01/03/2023	Cambio de formato	Modelo de evaluación externa 2021 con fines de acreditación para los Institutos Superior Técnicos y Tecnológicos
Versión 2	02/10/2023	02/10/2023	Contenido, Actividades colaborativas, autónomas, practicas, etc. Bibliografía básica/bibliografía complementaria Resultados de aprendizaje	Informe de seguimiento, control y evaluación curricular



Versión 3	17/09/2024	17/09/2024	Actualización de contenido de acuerdo a necesidad de abordar temas previos a UML	Informe de seguimiento, control y evaluación del proceso docente.
Versión 4	19/09/2025	19/09/2025	Actualización de contenido sobre la incorporación de metodologías de Desarrollo de Software Agiles.	Informe de seguimiento, control y evaluación del proceso docente.

9. TRABAJO FINAL DE ASIGNATURA

TRABAJO FINAL DE ASIGNATURA	
DATOS INFORMATIVOS	
TEMA:	Elaborar los diagramas UML de un sistema de reservación de hoteles
OBJETIVO GENERAL:	Elaborar los diagramas de (Caso de Usos, MER, Diagrama de Secuencias, Diagrama de Clases, Diagrama de Estados y Diagrama de Despliegue para un sistema de reservación de hoteles.
ESTRATEGIA/ TÉCNICA:	Aprendizaje Basado en Proyectos
PASOS / ACTIVIDADES:	• Objetivo Proporcione un producto construido junto con la documentación. Defina la arquitectura básica. Planifique el proyecto considerando los recursos disponibles. • Planificación Construya los diferentes diagramas UML para el sistema propuesto en el tema. Determine las relaciones existentes entre los diagramas. • Ejecución Realice la implementación de la arquitectura de los diagramas UML. • Evaluación Revisar la consistencia de los diferentes modelos de UML. Evalúe las pruebas.
BIBLIOGRAFÍA:	Booch, G., Rumbaugh, J., Jacobson, I. (2006). El Lenguaje Unificado de Modelado Uml 2.0 (2ª ed.). Pearson. (007.4035 - Libro Físico) Debrauwer, L., Evain, Y., Ferrandez, S. (2021). Design Patterns en Php. los 23 Patrones de Diseño: Descripciones y Soluciones Ilustradas en Uml 2 y Php (2ª ed.). Eni. (005.8069 - Libro Físico)

RÚBRICA DEL TRABAJO FINAL DE ASIGNATURA

INFORME/0.5:

CONSIDERACIONES	0,10 MUY BUENO	0,05 BUENO	0,03 REGULAR
Ortografía	No existen errores ortográficos en el informe	Se detecta un error ortográfico en el informe	Se detecta 2 o 3 errores ortográficos en el informe
Contenido	Toda la información en el informe es coherente con el tema planteado	El informe es coherente con el tema planteado.	No es coherente con el tema ni con el informe.
Proceso	Cumple proceso y validación del mismo	Cumple el proceso, pero no es válido.	No cumple el proceso
Método y técnicas de presentación	El método y las técnicas presentadas denotan seguridad.	El método y las técnicas presentadas denotan relativa seguridad.	El método y las técnicas presentadas denotan poca seguridad.
Originalidad en la presentación	Denota un alto grado de originalidad.	Denota medianamente originalidad.	Se denota poca originalidad del documento.

DESARROLLO-SEGUIMIENTO-SUSTENTACIÓN/0.5:

CONSIDERACIONES	0,10 MUY BUENO	0,05 BUENO	0,01 REGULAR
Volumen de voz	El volumen es lo suficientemente alto para ser escuchado por todos los miembros.	El volumen es cambiante a medida que avanza en la presentación	El volumen no es muy aceptable, es débil para ser escuchado por todos los miembros de la audiencia.
Postura del cuerpo y contacto visual	Siempre tiene buena postura y se proyecta seguro de sí mismo. Establece contacto visual con todos en el salón durante la presentación	Casi siempre tiene buena postura y establece contacto visual con todos en el salón durante la presentación.	Casi siempre tiene buena postura, pero no mira a las personas durante la presentación.
Vocalización	Habla claramente y es entendible.	Casi siempre habla claramente pero mientras avanza se pierde la claridad.	A veces habla entre dientes o no se le puede entender.
Conocimiento del tema	Demuestra un conocimiento completo del tema.	Demuestra un buen conocimiento del tema.	No parece conocer muy bien el tema.
Contestar preguntas	El estudiante puede con precisión contestar todas las preguntas planteadas sobre el tema	El estudiante puede con precisión contestar la mayoría de las preguntas planteadas sobre el tema	El estudiante puede contestar muy pocas preguntas, sin mucha precisión.

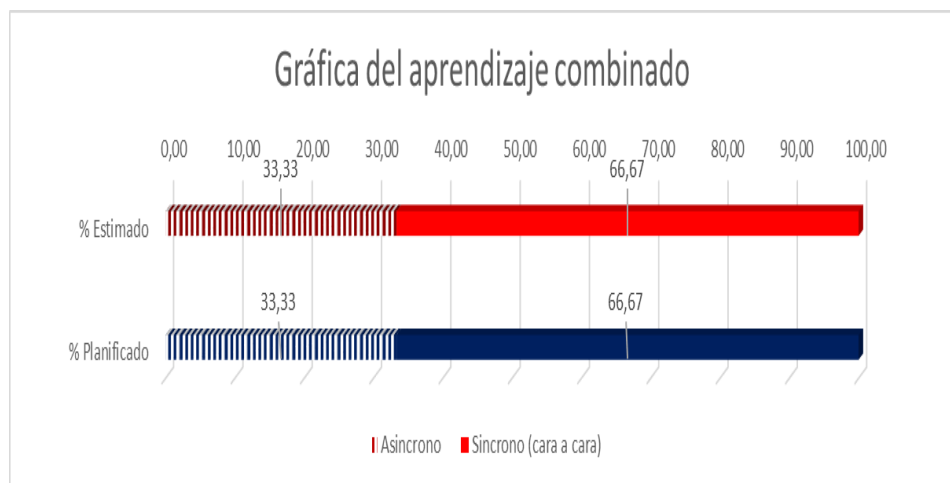


10. ORGANIZACIÓN DEL APRENDIZAJE:

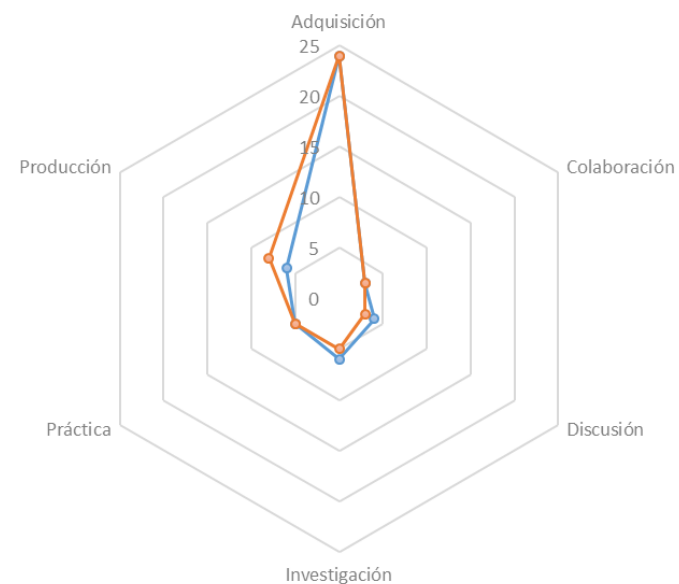
Descripción de la asignatura:

Aplicar patrones de diseño modernos para la construcción de una aplicación de software utilizando herramientas CASE de modelado y diagramación UML.

Gráfica del aprendizaje combinado



Tipos de aprendizaje estimados y planificados





1. Revisar los niveles de tipos de aprendizaje y de actividades por tipo de aprendizaje
2. De acuerdo al syllabus, asignar las actividades generadas que se van a ejecutar para abordar los temas de la Unidad
3. Seleccionar actividades para seleccionar entre varias tipos de aprendizaje (construccion) y valores del mismo tipo seleccionados de la Unidad abaja
4. Ingresar el tiempo estimado que los estudiantes dedicaran a esta actividad
5. Indicar el tipo de evaluacion (cualitativa/ cuantitativa) al que tributa la actividad; algunas pruebas se abren
6. Indicar los resultados de aprendizaje se logran con la actividad. Seleccionar valores resultados de aprendizaje en los tres siguientes

[illegible]



INSTITUTO SUPERIOR TECNOLÓGICO
VIDA NUEVA
SEDE MATRIZ



CARRERA:

TECNOLOGÍA SUPERIOR EN DESARROLLO DE SOFTWARE

ASIGNATURA: ANALISIS DE DISEÑO DE SOFTWARE

CÓDIGO: 24241041

MODALIDAD: PRESENCIAL

JORNADA: MATUTINA

NIVEL: CUARTO

TEMA:

PORTAFOLIO ASIGNATURA

AUTOR:

CISNEROS MONTOYA MATEO ESTEBAN

DOCENTE: ING. GAVILANES OBREGON KEVIN ALEXANDER MG.

MARZO 2026 – AGOSTO 2026

QUITO – ECUADOR



PLANTILLAS ABC

Plantillas ABC

Pasos: 3-4							Paso: 6						
TOTAL HORAS AUTÓNOMAS													
32,0	5,0	3,0	5,0	5,0	8,0	16,0							
Tiempo Prae	Discusión	Tiempo Dis	Investigación	Tiempo Inv	Producción	Tiempo Prod	Resultados de aprendizaje	DETALLE DE ACTIVIDADES DE ADQUISICIÓN	DETALLE DE ACTIVIDADES DE COLABORACIÓN	DETALLE DE ACTIVIDADES DE PRÁCTICAS	DETALLE DE ACTIVIDADES DE DISCUSIÓN	DETALLE DE ACTIVIDADES DE INVESTIGACIÓN	DETALLE DE ACTIVIDADES DE PRODUCCIÓN
1,0					Informes	1,0	Maneja las técnicas y el modelado UML previo al desarrollo de software basado en parámetros de calidad.	Clases de UML (Lenguaje Unificado de Modelado).	Simular el proceso de ingeniería de requisitos para un proyecto de software (identificar, documentar y clasificar los requisitos funcionales y no funcionales).	Desarrollo de la guía de práctica "UML".		Consulta bibliográfica respecto a las técnicas de levantamiento de requerimientos de software, realizar una tabla comparativa con los conceptos vistos en los videos.	Elaboración del informe de la guía práctica "UML".
1,0			Buscar y evaluar información e ideas.	1,0	Evaluación parcial	1,0	Maneja las técnicas y el modelado UML previo al desarrollo de software basado en parámetros de calidad.	Clases de elementos generales que utiliza UML para los diferentes diagramas en el modelamiento de software.	Modelar un caso de uso de una aplicación simple utilizando UML y organizar las tareas en un sprint ágil, asignando prioridades y responsables.	Realizar un levantamiento de procesos para el sistema de Tickets de soporte técnico en Visio / herramientas BPMN.		Investigación dando uso a ChatGPT respecto a las metodologías, técnicas y herramientas para modelar el proceso de software.	Evaluación de la unidad I respecto a UML.
1,0	Tutoriales	1,0	Utilice herramientas en línea para buscar y evaluar información e ideas.	1,0	Informes	1,0	Elabora diagramas UML en base a los requerimientos de usuario aplicando normas de procesos y calidad de software.	Clases de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) en la herramienta CASE Power Designer.	Desarrollar un organizador gráfico sobre los componentes de cada diagrama.	Desarrollo de la guía de práctica "Casos de Usos".	Observar el video tutorial relacionado al Diagrama de Caso de Usos. Link: https://drive.google.com/file/d/1edvLNGhNkV68Y23gHSjP0vicsF3TMEa/view?usp=sharing	Investigar la creación de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) en publicaciones glo buscadores académicos Link - Google Scholar: https://scholar.google.com/citations?hl=es&user=uniojia&asf=Solet https://scielo.org/psf	Elaboración del informe de la guía práctica "Casos de Usos".
1,0	Publicar y responder a foros de discusión asincrónicos.	1,0	Utilice herramientas en línea para buscar y evaluar información e ideas.	1,0	Informes	1,0	Efectúa el análisis de los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software Orientado a Objetos.	Clases de los diferentes diagramas UML (Diagrama de caso de usos, Modelo Entidad Relación, Diagrama de Clases, Diagrama de Secuencias, Diagrama de Transición de Estados) en la herramienta.		Desarrollo de la guía de práctica "Diagramas de Objetos".	Discutir a través de un foro individual en la plataforma institucional Moodle, sobre la Visión general de los diferentes diagramas UML. Link: www.canva.com	Elaboración del informe de la guía práctica "Diagramas de Objetos".	
1,0					Informes	1,0	Efectúa el análisis de los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software Orientado a Objetos.	Clases de la exportación: public, private, protected.	Desarrollar un mapa mental sobre las metodologías de desarrollo y los ciclos de vida del software, en la herramienta en línea mindomo Link: https://www.mindomo.com/es-es/crear-una-diapositiva	Desarrollo de la guía de práctica "Diagramas de Actividades".			Elaboración del informe de la guía práctica "Diagramas de Actividades".
					Evaluación parcial	1,0	Efectúa el análisis de los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software Orientado a Objetos.	Clases sobre los tipos de UML: Relaciones, Composición, Agregación, Asociación, Dependencia, Generalización, Realización.					Evaluación de la unidad II y III respecto a diagramas y vistas UML.
								Aplicación de la evaluación final de la asignatura					
			Buscar y evaluar información e ideas.	2,0	Proyectos	2,0			Sustentación del trabajo final de asignatura por grupos			Investigación de aspectos relacionados a la temática del trabajo final de asignatura	Elaboración del informe del proyecto final de asignatura
								Socialización de las notas finales de la asignatura			Aportar en un foro dentro de la plataforma sobre los temas de renovación		

COMPONENTES DE APRENDIZAJE							
UNIDADES DE ESTUDIO	En contacto con el docente	Práctico-experimental	Autónomo				
Uno	8	2	3				
Dos	6	1	3				
Tres	13	2	10				
Cuatro	#¡REF!	#¡REF!	#¡REF!				
Cinco	#¡REF!	#¡REF!	#¡REF!				
Seis	#¡REF!	#¡REF!	#¡REF!				
Siete	#¡REF!	#¡REF!	#¡REF!				
Ocho	#¡REF!	#¡REF!	#¡REF!				
Nueve	#¡REF!	#¡REF!	#¡REF!				
Diez	#¡REF!	#¡REF!	#¡REF!				

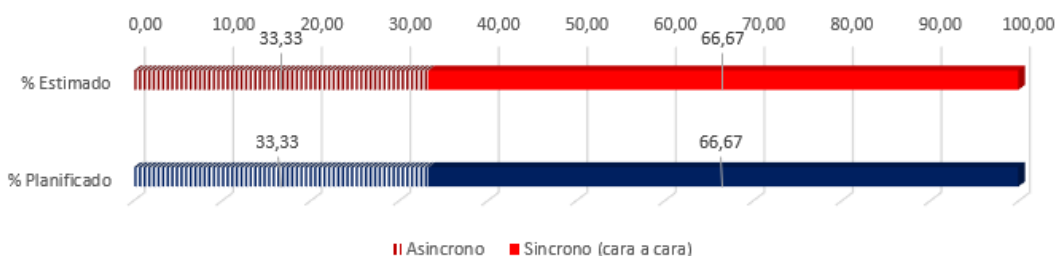
Etiquetas de fili	Suma de Tiempo Adq	Suma de Tiempo Col	Suma de Tiempo Prae	Suma de Tiempo Dis	Suma de Tiempo Inv	Suma de Tiempo Prod
1	7	1	2		1	2
2	6		1	1	1	1
3	11	2	2	2	3	5
(en blanco)						
Total general	24	3	5	3	5	8

Comparación entre el porcentaje estimado y real del trabajo sincrónico y asincrónico

Analice la diferencia entre los porcentajes que inicialmente se estimó y lo que realmente planificado en el diseño instruccional

Aprendizaje	% Estimado	% Planificado	Porcentaje de acuerdo al diseño instruccional
Asincrónico	33,33	33,33	
Sincrónico (cara a cara)	66,67	66,67	

Gráfica del aprendizaje combinado



Observe la estimación inicial que se realizó

Aprendizaje estimados	
Tipo de aprendizaje	Horas
Adquisición	24
Colaboración	3
Discusión	4
Investigación	6
Práctica	5
Producción	6
Total	48

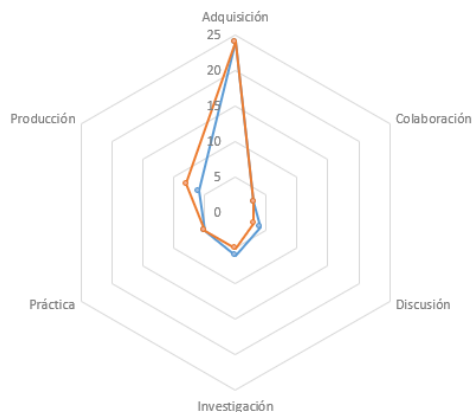
Nota: Las horas están asignadas directamente del paso 2

Observe las horas realmente asignadas de acuerdo al diseño instruccional

Aprendizaje planificados	
Tipo de aprendizaje	Horas
Adquisición	24
Colaboración	3
Discusión	3
Investigación	5
Práctica	5
Producción	8
Total	48

Nota: Las horas reales se completarán automáticamente según los planes para cada tema

Tipos de aprendizaje estimados y planificados



Nombre de la asignatura:	Análisis y Diseño de Software
Descripción de la asignatura: (280 caracteres)	Aplicar patrones de diseño modernos para la construcción de una aplicación de software utilizando herramientas CASE de modelado y diagramación UML.

Escriba los resultados de aprendizaje previstos por unidad de estudio	Maneja las técnicas y el modelado UML previo al desarrollo de software basado en parámetros de calidad.	2
	Elabora diagramas UML en base a los requerimientos de usuario aplicando normas de procesos y calidad de software.	1
	Efectúa el análisis de los requerimientos utilizando el análisis estructurado moderno y técnicas de modelamiento de software Orientado a Objetos.	3
		0
		0
		0
		0
		0

Observe el número de actividades que se asignó a cada resultado de aprendizaje de acuerdo al diseño instruccional planteado.



GUIAS PRACTICAS



GUÍA DE CLASES PRÁCTICAS

ASIGNATURA: ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA: TECNOLOGÍA SUPERIOR EN DESARROLLO DE SOFTWARE

MODALIDAD: PRESENCIAL

CÓDIGO DE LA ASIGNATURA: DSUP13

NIVEL: CUARTO

Elaborado por: Arias Martínez Darwin René



Índice	
Práctica No. 1	4
Código: DSUP13-GP-01	4
Título: Requerimientos de Software	4
Tiempo de duración: 120 minutos	4
Fundamentos:	4
Objetivos	5
Materiales y Herramientas	5
Normas de Seguridad	6
Preparación Previa del Estudiante	7
Procedimiento	9
Descripción de Resultados Obtenidos	9
Conclusiones	9
Evaluación del Aprendizaje	10
Bibliografía	11
Anexos	12
Práctica No. 2	17
Código: DSUP13-GP-02	17
Título: Diagrama Casos de Usos	17
Tiempo de duración: 120 minutos	17
Fundamentos:	17
Objetivos	18
Materiales y Herramientas	18
Normas de Seguridad	19
Preparación Previa del Estudiante	20
Procedimiento	21
Descripción de Resultados Obtenidos	21
Conclusiones	21
Evaluación del Aprendizaje	22
Bibliografía	23
Anexos	24
Práctica No. 3	26
Código: DSUP13-GP-03	26
Título: Diagrama Objetos	26
Tiempo de duración: 120 minutos	26
Fundamentos:	26
Objetivos	27
Materiales y Herramientas	28
Normas de Seguridad	28
Preparación Previa del Estudiante	29
Procedimiento	30



Descripción de Resultados Obtenidos	32
Conclusiones	32
Evaluación del Aprendizaje.....	32
Bibliografía	35
Anexos	36
Práctica No. 4	39
Código: DSUP13-GP-04	39
Título: Diagrama de actividades.....	39
Tiempo de duración: 120 minutos	39
Fundamentos:	39
Objetivos	40
Materiales y Herramientas	40
Normas de Seguridad.....	41
Preparación Previa del Estudiante	42
Procedimiento	43
Descripción de Resultados Obtenidos	45
Conclusiones.....	45
Evaluación del Aprendizaje.....	45
Bibliografía	48
Anexos	49



Práctica No. 1

Código: DSUP13-GP-01

Título: Requerimientos de Software

Tiempo de duración: 120 minutos

Fundamentos:

El análisis de requisitos de software es una etapa crucial en el ciclo de vida del desarrollo de software, ya que define lo que el sistema debe hacer y las restricciones bajo las cuales debe operar. Según Alarcón (2000), los requisitos de software son las necesidades y condiciones que un nuevo sistema debe cumplir. Estos requisitos pueden ser funcionales, describiendo las funciones específicas que el sistema debe realizar, y no funcionales, especificando criterios que pueden ser usados para juzgar la operación de un sistema, como el rendimiento, la seguridad y la usabilidad

El análisis de requisitos de software es una etapa crucial en el ciclo de vida del desarrollo de software, ya que define lo que el sistema debe hacer y las restricciones bajo las cuales debe operar. Según Alarcón (2000), los requisitos de software son las necesidades y condiciones que un nuevo sistema debe cumplir. Estos requisitos pueden ser funcionales, describiendo las funciones específicas que el sistema debe realizar, y no funcionales, especificando criterios que pueden ser usados para juzgar la operación de un sistema, como el rendimiento, la seguridad y la usabilidad.

El modelado de requisitos es otra técnica clave que ayuda a clarificar y documentar los requisitos. Se pueden utilizar diversos modelos, como diagramas de casos de uso, diagramas de actividades y diagramas de entidad-relación, que ayudan a visualizar cómo interactúan los componentes del sistema. Según Debrauwer, Ferrandez y Evain (2021), el uso de modelos permite una mejor comunicación entre el equipo de desarrollo y los stakeholders, asegurando que todos tengan una comprensión común del sistema.



Además, los modelos facilitan la identificación de requisitos incompletos, inconsistentes o contradictorios.

La validación y verificación de requisitos son pasos esenciales para asegurar que los requisitos definidos sean correctos y factibles. Debrauwer, Ferrandez y Evain (2021) subrayan que la validación se centra en revisar si los requisitos capturados realmente reflejan las necesidades de los stakeholders, mientras que la verificación asegura que los requisitos sean claros, sin ambigüedades y técnicamente realizables. La validación puede incluir revisiones, prototipos y simulaciones, mientras que la verificación puede involucrar la creación de casos de prueba para verificar que los requisitos se cumplen en el sistema final.

Objetivos

- Reconocer los requerimientos de software previo a la diagramación UML de un sistema académico.
- Identificar el tipo de requerimiento que se va a hacer uso el sistema.
(Funcionales, no funcionales).
- Definir un lenguaje de modelado visual fácil de aprender, pero enriquecido en significado.

Materiales y Herramientas

- Internet
- Software (StarUML)
- Computador



Normas de Seguridad

Uso adecuado del equipamiento

Además, deben manipular los dispositivos con cuidado, evitando movimientos bruscos y asegurándose de seguir las instrucciones específicas para el uso de hardware y software

Mantenimiento de la Seguridad Física

Los estudiantes deben mantener el área de trabajo ordenada y libre de obstáculos que puedan causar accidentes

Cumplimiento de las Políticas de la Institución

Respetar los horarios de uso del laboratorio y las normas de conducta establecidas por la institución.

Supervisión

Realizar todas las pruebas de funcionamiento del sistema bajo la supervisión del docente.

Inspección Previa

Inspeccionar visualmente todos los componentes antes de su uso para detectar posibles daños o defectos.



Preparación Previa del Estudiante

Fundamentos de Ingeniería de Software

Es esencial que los estudiantes tengan una comprensión básica de la ingeniería de software, incluyendo sus fases y procesos. Deben conocer el ciclo de vida del desarrollo de software (SDLC), que incluye etapas como la planificación, análisis, diseño, implementación, pruebas y mantenimiento. Este conocimiento proporciona un marco general que les ayuda a comprender dónde encaja el análisis de requisitos en el contexto más amplio del desarrollo de software.

Principios de Gestión de Proyectos

Antes de la clase práctica, los estudiantes deben leer y estudiar sobre el análisis de requisitos está estrechamente relacionado con la gestión de proyectos. Los estudiantes deben estar familiarizados con los principios básicos de gestión de proyectos, como la definición de alcance, la planificación de tareas, la asignación de recursos y el seguimiento del progreso. Herramientas y técnicas como diagramas de Gantt, metodologías ágiles y SCRUM también son útiles para coordinar y gestionar el proceso de recopilación y análisis de requisitos.

Técnicas de Recolección de Información

Es importante que los estudiantes sepan cómo recolectar información de diversas fuentes. Esto incluye habilidades en la realización de entrevistas, encuestas, talleres y grupos focales, así como en la observación directa y el análisis de documentos existentes. Estas técnicas son cruciales para obtener una comprensión clara y completa de los requisitos de los stakeholders.

Modelado y Documentación de Requisitos

Los estudiantes deberán conocer previamente sobre el modelado de requisitos. Esto incluye el uso de diagramas de casos de uso, diagramas de secuencia, diagramas de



actividad y otros modelos que ayudan a visualizar y comunicar los requisitos. Además, deben estar capacitados en la redacción de documentos de requisitos claros y precisos, utilizando plantillas y estándares de la industria, como IEEE 830.

Habilidades de Comunicación

La capacidad de comunicarse de manera efectiva con los stakeholders es fundamental en el análisis de requisitos. Los estudiantes deben desarrollar habilidades tanto verbales como escritas para poder interactuar con usuarios finales, clientes, desarrolladores y otros interesados de manera clara y efectiva. La empatía y la capacidad de escuchar activamente también son esenciales para comprender verdaderamente las necesidades y expectativas de los stakeholders.



Procedimiento

1. Analizar las características y requerimientos que el software debe cumplir. Ver anexo 1.
2. Identificar los requerimientos funcionales y no funcionales ver anexo (Descripción general del sistema), previo a la lectura de los conceptos.
3. Requerimientos funcionales y no funcionales. Ver anexo (Requerimientos del sistema).
4. Documentar los requerimientos funcionales y no funcionales identificados en la lectura del anexo (Descripción general del sistema). Y colocar en la tabla 1.
5. Describir los requerimientos no funcionales y no funcionales.
6. Analizar la mejor metodología de desarrollo de software que para el sistema académico, hable de cada una de sus etapas y escribir en la tabla 3.

Descripción de Resultados Obtenidos

Conclusiones



Evaluación del Aprendizaje

¿Cuáles son las principales diferencias entre requisitos funcionales y no funcionales, y por qué es importante identificar ambos tipos en un proyecto de software?

¿Qué técnicas de recolección de información son más efectivas para el análisis de requisitos y en qué contextos se recomienda utilizarlas?

¿Cómo pueden los diagramas de casos de uso y otros modelos visuales ayudar en el proceso de análisis de requisitos?

¿Cómo pueden los cambios en los requisitos afectar un proyecto de software y qué estrategias pueden implementarse para gestionar estos cambios de manera efectiva?



Criterio	Excelente (5)	Bueno (4)	Satisfactorio (3)	Necesita Mejora (2)	Insuficiente (1)
Análisis de Requisitos	Análisis completo y detallado, abarca todas las necesidades del usuario y stakeholders.	Análisis correcto con pequeñas omisiones en las necesidades del usuario.	Análisis funcional, pero con varios aspectos menores omitidos.	Análisis incompleto con errores significativos en las necesidades del usuario.	Análisis incorrecto o muy incompleto, no abarca las necesidades del usuario.
Diseño del Software	Diseño correcto y optimizado, cumple con todos los requisitos especificados.	Diseño correcto con pequeños errores, cumple con la mayoría de los requisitos.	Diseño funcional con varios errores menores, cumple con los requisitos básicos.	Diseño con errores significativos, cumple parcialmente con los requisitos.	Diseño incorrecto o incompleto, no cumple con los requisitos.
Modelado UML	Modelos UML completos y precisos, reflejan claramente la arquitectura del sistema.	Modelos UML correctos con pequeñas imprecisiones.	Modelos UML funcionales con varios errores menores.	Modelos UML con errores significativos, reflejan parcialmente la arquitectura del sistema.	Modelos UML incorrectos o muy incompletos.
Documentación Técnica	Documentación completa y detallada, fácil de entender y seguir.	Documentación completa y detallada, fácil de entender y seguir.	Documentación funcional, pero con varios errores menores.	Documentación incompleta con errores significativos	Documentación incorrecta o muy incompleta.
Verificación y Validación	Verificación y validación completas y precisas, cumplen con todos los requisitos y estándares.	Verificación y validación correctas con pequeños problemas.	Verificación y validación funcionales con varios problemas menores.	Verificación y validación con errores significativos, cumplen parcialmente con los requisitos y estándares.	Verificación y validación incorrectas o muy incompletas.

Bibliografía

Alarcón, R. (2000). *Diseño orientado a objetos con UML*. Madrid: Grupo EIDOS.

Booch, G., Rumbaugh, J., Jacobson, I., Martínez, J. S., & Molina, J. J. G. (2005). *El*

lenguaje unificado de modelado (Vol. 9). Madrid: Addison wesley.

Debrauwer, L., Ferrandez, S., Evain, Y. (2021). *Design patterns en php. Los 23*

patrones de diseño: descripciones y soluciones ilustradas en uml 2 y php (2th ed.). ENI Ediciones.



Anexos

Anexo 1.

Descripción general del sistema

Análisis General

Esta sección proporciona una visión detallada del sistema, precisando sus funciones, la información que maneja y todos los elementos implicados en su progreso:

Perspectivas del Producto

- **Objetivo de la Aplicación:** La aplicación está diseñada para ofrecer información general sobre la escuela y detalles específicos para cada tipo de usuario registrado.
- **Interfaz del Sistema:** Como es una aplicación web, puede ser utilizada en cualquier sistema operativo.
- **Interfaz de Usuario:** La interfaz es accesible para todos los usuarios; sin embargo, se necesita autenticarse como docente o administrador, el ingreso será mediante una intranet.
- **Interfaz de Hardware:** El hardware que posee la institución es suficiente para correr el sistema.
- **Interfaz de Software:** El sistema se ejecutará en cualquier dispositivo que tenga un navegador web moderno con acceso a internet
- **Interfaz de Comunicación:** Las comunicaciones se llevarán a cabo utilizando el protocolo HTTP sobre conexiones TCP/IP.
- **Restricción de Memoria:** La aplicación necesita un mínimo y espacio de disco

Funciones del Producto



El sistema está diseñado para asegurar un control seguro de los procesos académicos,

A continuación, se describen las funciones del sistema según los roles de los usuarios:

Usuario Docente

- **Alumnos Reporte:** Seleccionar el año lectivo y el año de básica para mostrar la lista de alumnos matriculados en el año seleccionado.
- **Registro de notas:** Seleccionar el trimestre y la asignatura para mostrar la lista de alumnos matriculados y registrar sus notas.
- **Registrar Faltas:** Seleccionar el año lectivo y registrar las faltas, mostrando las faltas justificadas e injustificadas de cada estudiante.
- **Modificar las notas:** Seleccionar el periodo y la asignatura para mostrar la lista de estudiantes matriculados y modificar sus notas.
- **Modificar Faltas:** Seleccionar el periodo para registrar las faltas, mostrando las faltas justificadas e injustificadas y permitiendo mantener o eliminar las faltas justificadas.
- **Listar Notas Reporte parcial:** Seleccionar el periodo, y mostrar un reporte con las asignaturas y las notas correspondientes.
- **Listar Notas:** Mostrar un reporte con las asignaturas y las notas correspondientes.
- **Reporte final:** Mostrar un reporte con las asignaturas y las notas finales.
- **Mostrar estudiantes Matriculados:** Seleccionar el tipo de matrícula para mostrar un reporte de los alumnos matriculados.
- **Mostrar Alumnos Aprobados:** Seleccionar el periodo, para mostrar un reporte de los alumnos aprobados.
- **Mostrar Alumnos Reprobados:** Mostrar un reporte de los alumnos reprobados.



- Mostrar asistencia final: Mostrar un reporte de asistencia de los alumnos.

Usuario Administrador

El usuario administrador tiene acceso total a todas las funcionalidades del sistema:

- Registrar nuevos usuarios.
- Registrar nuevas materias.
- Registrar nuevos años de básica.
- Crear periodo lectivo: Registrar nuevos años lectivos.
- Registrar nuevos docentes y asignarlos a un año de básica.
- Eliminar usuarios.
- Eliminar asignaturas.
- Eliminar años de básica.
- Eliminar años lectivos.
- Eliminar docentes.
- Actualizar la información de los usuarios.
- Actualizar la información de los años de básica.
- Actualizar la información de las asignaturas.
- Actualizar la información de los años lectivos.
- Actualizar la información de los docentes.
- Matricular alumnos en un año de básica después de ingresar los datos necesarios.
- Realizar cambios de paralelo para los alumnos.
- Anular los procesos realizados por los alumnos.
- Generar el proceso de aprobación de los alumnos.



Características del Usuario

Existen dos tipos de usuarios: no registrados y registrados.

- **Usuarios No Registrados:** Acceden únicamente a la información general de la institución publicada en el sitio web.
- **Usuarios Registrados:** Tienen acceso a la intranet, diferenciándose en dos subtipos:
 - **Usuario Docente:**
 - **Usuario Administrador:**

Anexo 2

Requerimientos del sistema

Requerimientos Funcionales

Los requisitos funcionales describen las funciones y características específicas que debe tener un sistema para cumplir con sus objetivos. Estos requisitos detallan qué debe hacer el sistema, cómo debe comportarse y las interacciones entre los usuarios y el sistema. Incluyen especificaciones sobre procesos, datos, interfaces y reportes necesarios para garantizar el correcto funcionamiento del software según las necesidades del usuario.

Requisitos no Funcionales

Los requisitos no funcionales definen los criterios de calidad y las restricciones del sistema que no están directamente relacionados con comportamientos específicos. Estos incluyen aspectos como la seguridad, el rendimiento, la usabilidad, la fiabilidad



y la escalabilidad. Los requisitos no funcionales aseguran que el sistema sea eficiente, seguro y fácil de mantener, proporcionando una experiencia de usuario satisfactoria y cumpliendo con las expectativas de calidad.



Práctica No. 2

Código: DSUP13-GP-02

Título: Diagrama Casos de Usos

Tiempo de duración: 120 minutos

Fundamentos:

Los diagramas de UML (Unified Modeling Language) son una herramienta esencial en el análisis y diseño de software, ya que proporcionan una representación visual de los componentes y las relaciones dentro de un sistema. Según Booch, Rumbaugh, y Jacobson (2005), UML es un lenguaje de modelado estándar ampliamente utilizado para especificar, visualizar, construir y documentar los artefactos de sistemas de software. UML permite a los desarrolladores y analistas crear modelos que son tanto comprensibles para los stakeholders como detallados para los ingenieros de software, facilitando así una comunicación efectiva y una planificación precisa del desarrollo del sistema.

Entre los diversos tipos de diagramas UML, los diagramas de casos de uso son particularmente importantes para capturar y comunicar los requisitos funcionales de un sistema. Los diagramas de casos de uso, introducidos por Debrauwer, Ferrandez y Evain (2021), describen las interacciones entre los actores (usuarios u otros sistemas) y el sistema en cuestión, especificando las funciones que el sistema debe realizar. Estos diagramas son útiles no solo para los desarrolladores, sino también para los stakeholders no técnicos, ya que presentan una visión clara y comprensible de cómo se utilizará el sistema.

El uso de diagramas de casos de uso en el análisis de requisitos ayuda a identificar y organizar los requisitos funcionales del sistema de manera estructurada. Según Alarcón



(2000), los casos de uso permiten descomponer un sistema en partes manejables, facilitando la comprensión de los requisitos y la identificación de posibles problemas o áreas de mejora. Además, los diagramas de casos de uso ayudan a establecer el alcance del proyecto y a priorizar las funcionalidades que deben ser implementadas, proporcionando una base sólida para la planificación y gestión del desarrollo del software.

Finalmente, UML no se limita a los diagramas de casos de uso. Incluye otros tipos de diagramas, como los diagramas de clases, secuencia, actividades y componentes, cada uno con un propósito específico en el ciclo de vida del desarrollo de software. Como señalan Debrauwer, Ferrandez y Evain (2021), estos diagramas permiten una descripción detallada de la estructura y comportamiento del sistema, desde la conceptualización inicial hasta el diseño detallado y la implementación. En conjunto, UML y los diagramas de casos de uso ofrecen un enfoque integral para el análisis y diseño de software, asegurando que los sistemas desarrollados sean robustos, escalables y alineados con las necesidades de los usuarios.

Objetivos

- Realizar diagramas UML y de casos de uso respetando las normas.
- Identificar los actores y elementos que conforman cada uno de estos diagramas.
- Diagramar en el software correspondiente los diagramas que se solicitan y comprobar la secuencia.

Materiales y Herramientas

- Internet
- Software (StarUML)
- Computador



Normas de Seguridad

Uso adecuado del equipamiento

Además, deben manipular los dispositivos con cuidado, evitando movimientos bruscos y asegurándose de seguir las instrucciones específicas para el uso de hardware y software

Mantenimiento de la Seguridad Física

Los estudiantes deben mantener el área de trabajo ordenada y libre de obstáculos que puedan causar accidentes

Cumplimiento de las Políticas de la Institución

Respetar los horarios de uso del laboratorio y las normas de conducta establecidas por la institución.

Supervisión

Realizar todas las pruebas de funcionamiento del sistema bajo la supervisión del docente.



Preparación Previa del Estudiante

Fundamentos de Programación y Conceptos de Orientación a Objetos.

Es esencial que los estudiantes tengan una sólida comprensión de los fundamentos de la programación y los conceptos de orientación a objetos (OO). Esto incluye conocimientos sobre clases, objetos, herencia, polimorfismo y encapsulación. Estos conceptos son la base de muchos de los diagramas UML, especialmente los diagramas de clases y objetos. Un buen dominio de un lenguaje de programación orientado a objetos, como Java o C++, también es altamente beneficioso.

Introducción a la Ingeniería de Software.

Los estudiantes deben estar familiarizados con los principios básicos de la ingeniería de software, incluyendo el ciclo de vida del desarrollo de software (SDLC) y las diversas fases que lo componen, como el análisis, diseño, implementación, pruebas y mantenimiento. Este conocimiento proporciona un marco de referencia para entender cómo UML se integra en el proceso de desarrollo de software y cómo se utilizan sus diagramas en cada fase.

Análisis de Requisitos

Es indispensable que los estudiantes entiendan cómo se recopilan y analizan los requisitos de un sistema. Esto incluye técnicas para la recolección de requisitos, como entrevistas, encuestas y análisis de documentos, así como la capacidad de documentar y priorizar estos requisitos. Este conocimiento es fundamental para crear diagramas de casos de uso, que son una herramienta clave para representar los requisitos funcionales del sistema.

Modelado de Sistemas

Tener experiencia previa en modelado de sistemas, aunque sea a nivel básico, es muy útil. Esto puede incluir el uso de diagramas de flujo de datos (DFD), diagramas



entidad-relación (ERD) y otros modelos que representan sistemas de manera visual. Esta experiencia ayuda a los estudiantes a adaptarse más fácilmente al uso de UML, que también utiliza diagramas para representar distintos aspectos de los sistemas.

Procedimiento

1. Analizar conceptos a través de la lectura y conocer la importancia. Ver anexo (Caso de Usos).
2. Graficar los diagramas de casos de uso y el de secuencia
3. previo al análisis de las especificaciones de casos de usos del anexo (Especificaciones de Caso de Usos).
4. Describir de forma individual cada diagrama y grafica. Tomando en cuenta:
 - a. Actor que inicia.
 - b. Condiciones previas.
 - c. Pasos.
 - d. Condiciones resultantes.
 - e. Actor beneficiario.
5. Documentar los actores que intervienen en flujo del software y registrar en la

Descripción de Resultados Obtenidos

Conclusiones



Evaluación del Aprendizaje

¿Cuál es la diferencia entre un actor primario y un actor secundario en un diagrama de casos de uso, y cómo afectan estas diferencias al diseño del sistema?

¿Cómo se identifican los casos de uso en un sistema complejo, y qué técnicas pueden utilizarse para asegurarse de que todos los requisitos funcionales estén representados?

Explique cómo un diagrama de casos de uso puede ayudar a definir el alcance de un proyecto de software y facilitar la comunicación entre stakeholders y el equipo de desarrollo.

¿Qué es una relación de inclusión y una relación de extensión en un diagrama de casos de uso, y cuándo se deben usar cada una de estas relaciones?



Criterio	Excelente (5)	Bueno (4)	Satisfactorio (3)	Necesita Mejora (2)	Insuficiente (1)
Identificación de Actores	Todos los actores relevantes están identificados y correctamente etiquetados.	La mayoría de los actores relevantes están identificados, con algunos menores faltantes.	Actores principales identificados, pero faltan varios secundarios importantes.	Varios actores relevantes faltan o están incorrectamente etiquetados.	La identificación de actores es incorrecta o está muy incompleta.
Identificación de Casos de Uso	Todos los casos de uso relevantes están de manera clara definidos y correctamente etiquetados.	La mayoría de los casos de uso están definidos, con algunos casos menores faltantes o incorrectos.	Casos de uso principales definidos, pero faltan varios menores importantes.	Varios casos de uso importantes faltan o están incorrectamente definidos.	La identificación de casos de uso es incorrecta o está muy incompleta.
Relaciones entre Casos de Uso	Relaciones de inclusión y extensión correctamente aplicadas y claramente indicadas.	Relaciones correctamente aplicadas en su mayoría, con algunos errores menores.	Relaciones de inclusión y extensión aplicadas, pero con varios errores o omisiones.	Relaciones de inclusión y extensión aplicadas incorrectamente o faltantes en varios casos.	Las relaciones entre casos de uso son incorrectas o están ausentes.
Claridad y Comprensibilidad	El diagrama es claro y fácilmente comprensible por todos los stakeholders.	El diagrama es mayormente claro, con algunas áreas que podrían mejorarse.	El diagrama es funcional, pero con varias áreas confusas o difíciles de entender.	El diagrama es difícil de entender en varias áreas importantes.	El diagrama es confuso y difícil de entender en su totalidad.
Complejidad y Detalle	El diagrama cubre todos los aspectos relevantes del sistema con un nivel de detalle adecuado.	El diagrama cubre la mayoría de los aspectos relevantes, con algunos detalles menores faltantes.	El diagrama cubre los aspectos principales, pero varios detalles importantes están faltando.	El diagrama cubre parcialmente el sistema, con muchos detalles importantes faltantes.	El diagrama está muy incompleto, faltando muchos aspectos y detalles relevantes.

Bibliografía

Alarcón, R. (2000). *Diseño orientado a objetos con UML*. Madrid: Grupo EIDOS.

Booch, G., Rumbaugh, J., Jacobson, I., Martínez, J. S., & Molina, J. J. G. (2005). *El lenguaje unificado de modelado* (Vol. 9). Madrid: Addison wesley.



Debrauwer, L., Ferrandez, S., Evain, Y. (2021). *Design patterns en php. Los 23 patrones de diseño: descripciones y soluciones ilustradas en uml 2 y php* (2th ed.). ENI Ediciones.

Anexos

Anexo 1

Definición de un diagrama de casos de uso

Un caso de uso refiere una serie de acciones entre un sistema y sus actores en respuesta a un evento iniciado por un actor. Los diagramas de casos de uso se utilizan para detallar la comunicación y el comportamiento de un sistema a través de su interacción con usuarios u otros sistemas. En esencia, son diagramas que muestran la relación entre los actores y los casos de uso en un sistema.

En este contexto, es un vínculo entre los elementos del modelo, como la especialización y la generalización. Este tipo de diagramas se utilizan para representar los requisitos del sistema, demostrando cómo responde a los eventos que se producen dentro de su entorno.

Los casos de uso son fundamentales en el proceso de modelado de sistemas, particularmente desde la perspectiva del paradigma orientado a objetos, comprendiendo el análisis y diseño orientados a objetos. Estos casos constituyen parte del Lenguaje Unificado de Modelado.

La principal aplicación de los casos de uso se encuentra en el análisis y diseño del sistema, especialmente en la definición de los requisitos del usuario. Son una herramienta de comunicación eficaz debido a la simplicidad en su creación y comprensión.



Anexo 2 Especificaciones

ESPECIFICACIONES DE CASOS DE USOS

Código	1		
Nombre	Ingresar al sistema		
Descripción	Permite el ingreso de un usuario al sistema		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, Docentes, sistema		
Precondición	El usuario administrador o Docente debe tener el usuario y password válida para poder acceder al sistema.		
Pos condición	El administrador puede acceder al sistema		
Flujo normal	1. Ingresar nombre de usuario 2. Ingresar password 3. Validar datos		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	2		
Nombre	Año de básica		
Descripción	Permite altas, bajas, modificaciones de un año de básica.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Asignar asignaturas a un año de básica		
Pos condición	Los datos son almacenados en la base de datos		
Flujo normal	1. Ingresar campos obligatorios 2. Validar datos 3. Alta de año de básica		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	3		
Nombre	Asignatura		
Descripción	Permite altas, bajas, modificaciones de una asignatura.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Debe ingresar todos los campos obligatorios para dar de alta una asignatura.		
Pos condición	Los datos son almacenados en la base de datos		
Flujo normal	1. Ingresar campos obligatorios 2. Validar datos 3. Alta de año de básica		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	4		
Nombre	Matrícula		
Descripción	Permite generar matrícula del alumno.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Asignar un año de básica al alumno.		
Pos condición	Imprime ficha de matrícula		
Flujo normal	1. Identificar alumnos antiguos y nuevos 2. Validar datos 3. Genera matrícula		
Excepciones	1. Ingresar al sistema 2. Cambiar valores viejos por nuevos 3. Guardar		
Anotaciones	Ninguna		

Código	5		
Nombre	Reporte trimestre, anuales		
Descripción	Este caso de uso permite realizar un reporte de todas las parciales de notas y asistencia.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, Docente		
Precondición	Validar datos del administrador en el sistema		
Pos condición	Debe imprimirse un informe		
Flujo normal	1. Ingresar al sistema 2. validar datos del administrador, Docente 3. generar reporte 4. imprimir		
Excepciones	1. Ingresar al sistema 2. modificar notas 3. Guardar		
Anotaciones	Ninguna		

Código	6		
Nombre	Docente		
Descripción	Los docentes son los encargados de registrar las notas.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Docente, sistema		
Precondición	El usuario Docente debe tener el usuario y password válida para poder acceder al sistema.		
Pos condición	El docente puede acceder al sistema		
Flujo normal	1. Ingresar nombre de usuario 2. Ingresar password 3. Validar datos		
Excepciones	Ninguna		
Anotaciones	Genera reporte de alumnos Genera reporte de notas bimestrales		



Práctica No. 3

Código: DSUP13-GP-03

Título: Diagrama Objetos

Tiempo de duración: 120 minutos

Fundamentos:

Los diagramas de objetos de UML (Unified Modeling Language) son una herramienta fundamental en el análisis y diseño de sistemas orientados a objetos. Estos diagramas proporcionan una representación instantánea de los objetos en un sistema y sus relaciones en un momento específico. A diferencia de los diagramas de clases, que muestran la estructura estática de un sistema, los diagramas de objetos representan instancias específicas de clases y los vínculos entre ellas en un contexto particular.

Booch, Rumbaugh, y Jacobson (2005), los creadores originales de UML consideran los diagramas de objetos como fundamentales para visualizar, especificar, construir y documentar los componentes de sistemas de software. Estos diagramas permiten a los desarrolladores comprender la interacción entre los objetos en un escenario específico y observar el comportamiento de las instancias de las clases en situaciones dinámicas. Este enfoque visual ayuda a identificar posibles problemas de manera más efectiva y la verificación de la consistencia del modelo en etapas tempranas del desarrollo.

Un aspecto clave de los diagramas de objetos es su capacidad para representar la estructura temporal del sistema. Según Debrauwer, Ferrandez y Evain (2021), estos diagramas son particularmente útiles para ilustrar ejemplos concretos de cómo se ven las instancias de clases en un escenario real, proporcionando una "fotografía" del estado del sistema en un momento dado. Esta representación es invaluable para validar que el diseño



de clases previsto funcionará como se espera en situaciones prácticas y para identificar y corregir problemas de diseño antes de que se implementen en el código.

Además, los diagramas de objetos pueden servir como un puente entre los análisis de requisitos y el diseño detallado del sistema. Como señalan Debrauwer, Ferrandez y Evain (2021), al trabajar con diagramas de objetos, los desarrolladores pueden comparar el comportamiento real de las instancias de clase con los requisitos especificados. Esto asegura que las abstracciones del diseño de clase sean correctas y que las interacciones entre objetos reflejen adecuadamente los escenarios de uso previstos.

Los diagramas de objetos también son útiles para documentar el comportamiento de los sistemas en escenarios de prueba y depuración. Alarcón (2000) destaca que estos diagramas pueden representar cómo las instancias de objetos cambian de estado en respuesta a eventos específicos, proporcionando un medio visual para rastrear y comprender el control y la evolución del estado del software. Esta capacidad es crucial para la depuración y para garantizar que el sistema efectúa con los requisitos funcionales y de rendimiento.

Objetivos

- Realizar los diagramas de clases y los diagramas de objetos de UML
- Identificar los elementos actores y componentes que intervienen para la creación de cada uno de los diagramas
- Aplicar los diagramas a través de un software que permita documentar esta información.



Materiales y Herramientas

- Internet
- Software (StarUML)
- Computador

Normas de Seguridad

Uso adecuado del equipamiento

Además, deben manipular los dispositivos con cuidado, evitando movimientos bruscos y asegurándose de seguir las instrucciones específicas para el uso de hardware y software

Mantenimiento de la Seguridad Física

Los estudiantes deben mantener el área de trabajo ordenada y libre de obstáculos que puedan causar accidentes

Cumplimiento de las Políticas de la Institución

Respetar los horarios de uso del laboratorio y las normas de conducta establecidas por la institución.

Supervisión

Realizar todas las pruebas de funcionamiento del sistema bajo la supervisión del docente.



Preparación Previa del Estudiante

Bases de Programación Orientada a Objetos (OOP)

Es fundamental que los estudiantes comprendan las bases de programación orientada a objetos. Esto incluye la familiaridad con términos y conceptos como clases, objetos, herencia, polimorfismo, encapsulación y abstracción. Un buen dominio de estos conceptos es esencial para entender cómo los objetos se relacionan e interactúan en un sistema.

Fundamentos de Ingeniería de Software

Los estudiantes deben tener una comprensión básica de la ingeniería de software, incluyendo el ciclo de vida del desarrollo de software (SDLC) y las diversas fases que lo componen. Este conocimiento proporciona un marco de referencia para entender cómo los diagramas de objetos se integran en el proceso de desarrollo de software.

Diagramas de Clases UML

Dado que los diagramas de objetos son instancias específicas de los diagramas de clases, es crucial que los estudiantes sepan cómo leer y crear diagramas de clases UML. Los diagramas de clases muestran la estructura estática del sistema, definiendo las clases, sus atributos, métodos y las relaciones entre ellas. Este conocimiento es fundamental para visualizar cómo las instancias de clase se representarán en los diagramas de objetos.

Conocimiento Básico de Bases de Datos

Una comprensión básica de bases de datos y SQL es beneficiosa, especialmente cuando se trabaja con diagramas de objetos que incluyen instancias de datos. Este conocimiento ayuda a los estudiantes a entender cómo los objetos persistirán en una base de datos y cómo las relaciones entre objetos pueden mapearse a relaciones entre tablas en una base de datos relacional.



Procedimiento

1. Analizar los respectivos diagramas de entidad relación y de clases de un sistema académico. Ver anexo (Diagrama de Clases).
2. Enumerar y describir las clases que se usaran para la construcción del diagrama de objetos y registrarlo en la tabla 1, previo al análisis del anexo *“Diagrama de clases”*.
3. Elaborar el diagrama de objetos de un sistema académico previo al análisis de la documentación que se encuentra en el anexo (Diagrama de Clases), graficar en tabla 2.
4. Elaborar una tabla de diferencias entre un diagrama de clases y de objetos. Colocar en la tabla 3.

Tabla 1

Clases

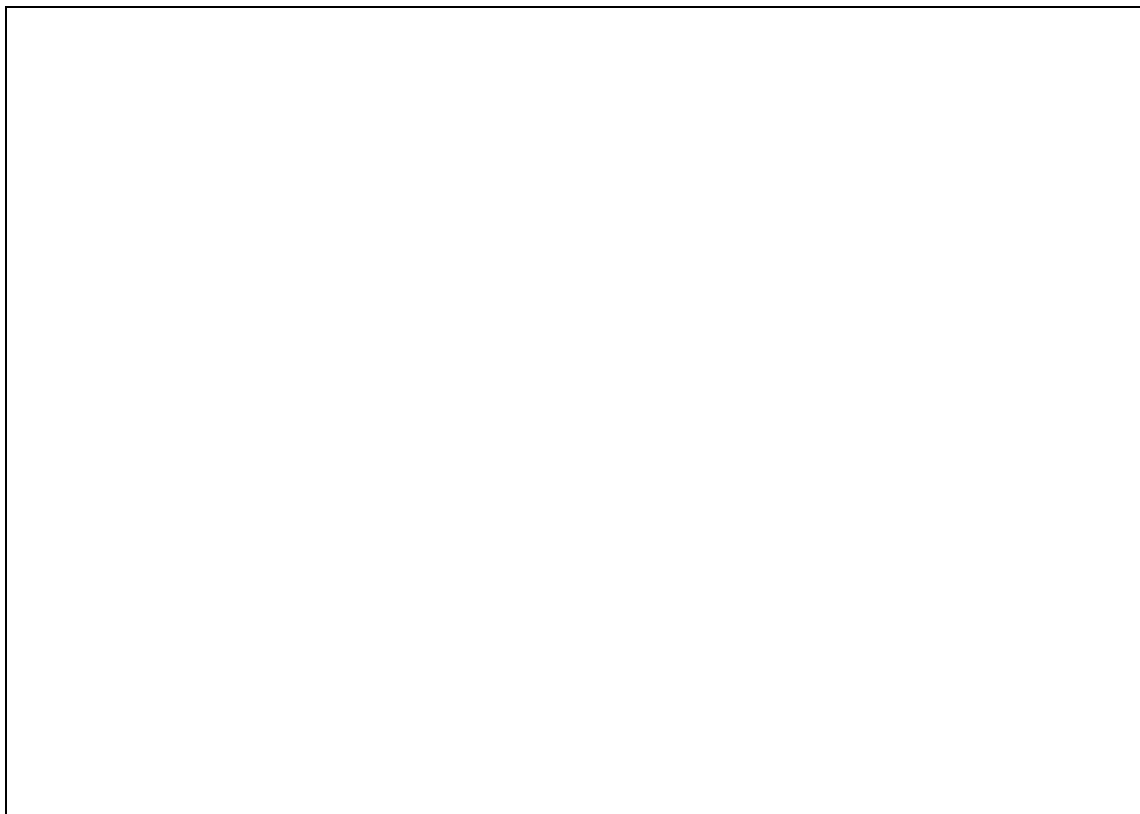
N.-	CLASES	DESCRIPCIÓN

Nota. Listado de clases a utilizar en el sistema



Tabla 2

Diagrama de objetos.



Nota. Diagrama de un sistema académico

Tabla 3

Tipos de diagramas

DIAGRAMA DE CLASES	DIAGRAMA DE OBJETOS

Nota. UML Sistema académico



Descripción de Resultados Obtenidos

Conclusiones

Evaluación del Aprendizaje

¿Escriba la diferencia entre un diagrama de clases y un diagrama de objetos en UML, y en qué situaciones es más apropiado utilizar cada uno?

Explique cómo los diagramas de objetos pueden ser utilizados para validar los requisitos de un sistema. Proporcione un ejemplo específico de cómo un diagrama de objetos puede revelar inconsistencias en el diseño.

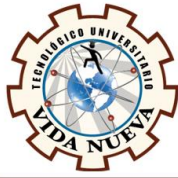


Describa el proceso de creación de un diagrama de objetos a partir de un diagrama de clases. ¿Qué información adicional necesita y cómo se representan las instancias de los objetos?

¿Cómo pueden los diagramas de objetos ayudar en la depuración y pruebas de un sistema de software? Proporcione ejemplos de cómo un diagrama de objetos puede ser utilizado para identificar errores o verificar el comportamiento del sistema.



Criterio	Excelente (5)	Bueno (4)	Satisfactorio (3)	Necesita Mejora (2)	Insuficiente (1)
Identificación de Objetos	Todos los objetos relevantes están identificados y correctamente etiquetados.	La mayoría de los objetos relevantes están identificados, con algunos menores faltantes.	Objetos principales identificados, pero faltan varios secundarios importantes.	Varios objetos relevantes faltan o están incorrectamente etiquetados.	La identificación de objetos es incorrecta o está muy incompleta.
Representación de Relaciones	Todas las relaciones entre objetos están correctamente representadas y etiquetadas.	La mayoría de las relaciones están representadas correctamente, con algunos errores menores.	Relaciones principales representadas, pero faltan varias relaciones menores importantes.	Varias relaciones relevantes faltan o están incorrectamente representadas.	La representación de relaciones es incorrecta o está muy incompleta.
Complejidad y Detalle	El diagrama cubre todos los aspectos relevantes del sistema con un nivel de detalle adecuado.	El diagrama cubre la mayoría de los aspectos relevantes, con algunos detalles menores faltantes.	El diagrama cubre los aspectos principales, pero varios detalles importantes están faltando.	El diagrama cubre parcialmente el sistema, con muchos detalles importantes faltantes.	El diagrama está muy incompleto, faltando muchos aspectos y detalles relevantes.
Consistencia con el Diagrama de Clases	El diagrama de objetos es completamente consistente con el diagrama de clases correspondiente.	El diagrama de objetos es mayormente consistente con el diagrama de clases, con algunos errores menores.	El diagrama de objetos tiene varios errores de consistencia con el diagrama de clases.	El diagrama de objetos es parcialmente consistente con el diagrama de clases, con muchos errores significativos.	El diagrama de objetos es inconsistente con el diagrama de clases o muy incompleto.
Conformidad con Estándares UML	El diagrama sigue todas las normas y convenciones UML de manera precisa.	El diagrama sigue las normas y convenciones UML con algunas desviaciones menores.	El diagrama sigue las normas UML en general, pero con varias desviaciones.	El diagrama sigue las normas UML de manera inconsistente, con muchas desviaciones importantes.	El diagrama no sigue las normas y convenciones UML correctamente.



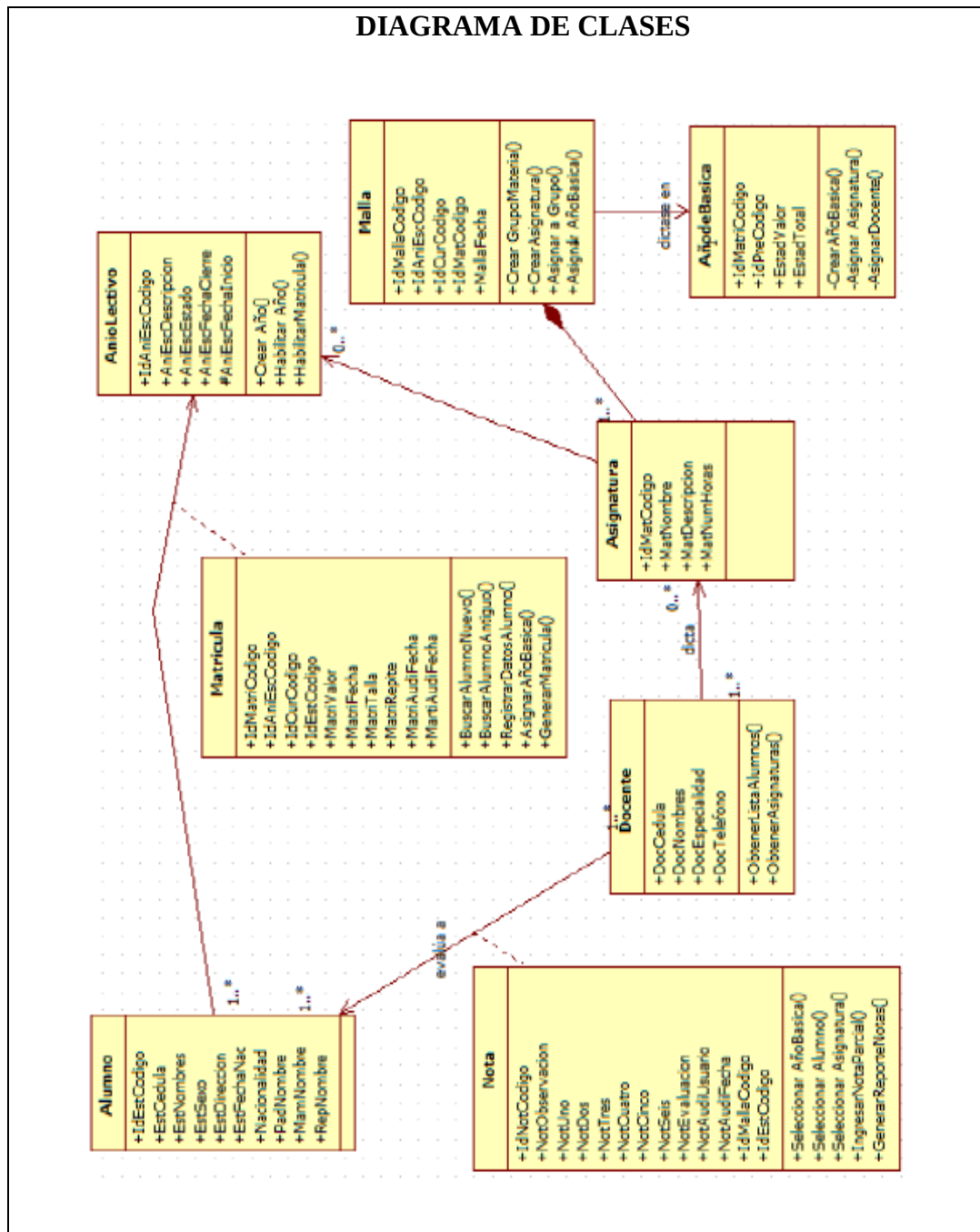
Bibliografía

- Alarcón, R. (2000). *Diseño orientado a objetos con UML*. Madrid: Grupo EIDOS.
- Booch, G., Rumbaugh, J., Jacobson, I., Martínez, J. S., & Molina, J. J. G. (2005). *El lenguaje unificado de modelado (Vol. 9)*. Madrid: Addison wesley.
- Debrauwer, L., Ferrandez, S., Evain, Y. (2021). *Design patterns en php. Los 23 patrones de diseño: descripciones y soluciones ilustradas en uml 2 y php* (2th ed.). ENI Ediciones.

Anexos

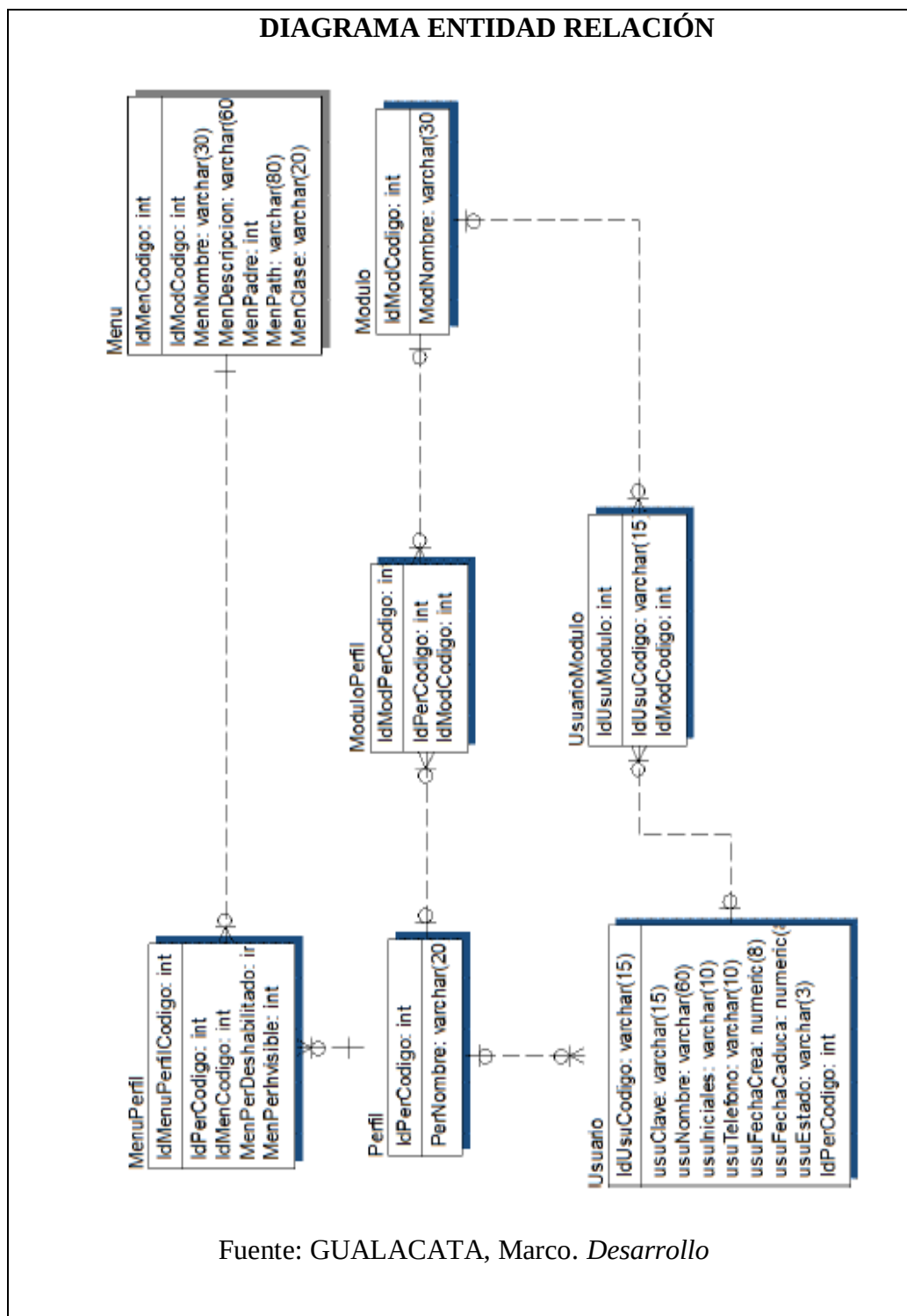
Figura 1

Diagrama de clases



Nota. En la imagen se puede observar el diagrama de clases con sus atributos.

Figura 1
Diagrama Entidad Relación



Nota. En la imagen se puede observar el diagrama E-R.



Figura 3

Especificaciones

CASOS DE USOS

Código	1		
Nombre	Ingresar al sistema		
Descripción	Permite el ingreso de un usuario al sistema		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, Docentes, sistema		
Precondición	El usuario administrador o Docente debe tener el usuario y password válida para poder acceder al sistema.		
Pos condición	El administrador puede acceder al sistema		
Flujo normal	1. Ingresar nombre de usuario 2. Ingresar password 3. Validar datos		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	2		
Nombre	Año de básica		
Descripción	Permite altas, bajas, modificaciones de un año de básica.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Asignar asignaturas a un año de básica		
Pos condición	Los datos son almacenados en la base de datos		
Flujo normal	1. Ingresar campos obligatorios 2. Validar datos 3. Alta de año de básica		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	3		
Nombre	Asignatura		
Descripción	Permite altas, bajas, modificaciones de una asignatura.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Debe ingresar todos los campos obligatorios para dar de alta una asignatura.		
Pos condición	Los datos son almacenados en la base de datos		
Flujo normal	1. Ingresar campos obligatorios 2. Validar datos 3. Alta de año de básica		
Excepciones	Ninguna		
Anotaciones	Ninguna		

Código	4		
Nombre	Matrícula		
Descripción	Permite generar matrícula del alumno.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, sistema		
Precondición	Asignar un año de básica al alumno.		
Pos condición	Imprime ficha de matrícula		
Flujo normal	1. Identificar alumnos antiguos y nuevos 2. Validar datos 3. Genera matrícula		
Excepciones	1. Ingresar al sistema 2. Cambiar valores viejos por nuevos 3. Guardar		
Anotaciones	Ninguna		

Código	5		
Nombre	Reporte trimestre, anuales		
Descripción	Este caso de uso permite realizar un reporte de todas las parciales de notas y asistencia.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Administrador, Docente		
Precondición	Validar datos del administrador en el sistema		
Pos condición	Debe imprimirse un informe		
Flujo normal	1. Ingresar al sistema 2. validar datos del administrador, Docente 3. generar reporte 4. imprimir		
Excepciones	1. Ingresar al sistema 2. modificar notas 3. Guardar		
Anotaciones	Ninguna		

Código	6		
Nombre	Docente		
Descripción	Los docentes son los encargados de registrar las notas.		
Fecha creación	Sept 2011	Fecha última modificación	Sept 16-2011
Actores	Docente, sistema		
Precondición	El usuario Docente debe tener el usuario y password válida para poder acceder al sistema.		
Pos condición	El docente puede acceder al sistema		
Flujo normal	1. Ingresar nombre de usuario 2. Ingresar password 3. Validar datos		
Excepciones	Ninguna		
Anotaciones	Genera reporte de alumnos Genera reporte de notas bimestrales		



Práctica No. 4

Código: DSUP13-GP-04

Título: Diagrama de actividades

Tiempo de duración: 120 minutos

Fundamentos:

Los diagramas de actividades en UML (Unified Modeling Language) son una herramienta visual fundamental para modelar el flujo de trabajo o actividades de un sistema. Según Booch, Rumbaugh y Jacobson (2005), estos diagramas ilustran las secuencias de actividades y el flujo de control entre ellas, proporcionando una vista detallada de los procesos dinámicos dentro de un sistema. Los diagramas de actividades son especialmente útiles para capturar procesos de negocio y casos de uso complejos, ayudando a los desarrolladores y analistas a comprender y comunicar el comportamiento de un sistema en términos operativos.

Un aspecto clave de los diagramas de actividades es su capacidad para representar tanto flujos de trabajo secuenciales como concurrentes. Según Alarcón (2000), los diagramas de actividades permiten modelar situaciones en las que múltiples actividades ocurren simultáneamente, utilizando nodos de bifurcación y unión para gestionar estos flujos paralelos. Esta característica es crucial para representar sistemas que dependen de operaciones concurrentes, como sistemas de procesamiento de transacciones o aplicaciones distribuidas, donde la sincronización y la coordinación entre diferentes actividades son fundamentales.

Además, los diagramas de actividades son valiosos para la validación y verificación de procesos. Alarcón (2000) señala que estos diagramas pueden ser utilizados para identificar y resolver posibles cuellos de botella y puntos de fallo en el flujo de trabajo, antes de que se implementen en el código. Al proporcionar una representación



clara y detallada del flujo de actividades, los diagramas de actividades facilitan la identificación de problemas de lógica y ayudan a garantizar que todos los escenarios posibles están considerados y gestionados adecuadamente.

Por último, los diagramas de actividades también juegan un papel importante en la documentación y la comunicación de procesos de negocio. Debrauwer, Ferrandez y Evain (2021) destacan que estos diagramas pueden ser utilizados para comunicar de manera efectiva los detalles operativos a los stakeholders no técnicos, como gerentes de negocio y usuarios finales. Al ofrecer una representación visual intuitiva, los diagramas de actividades mejoran la comprensión y la colaboración entre los equipos técnicos y no técnicos, asegurando que todos los involucrados tienen una comprensión compartida del funcionamiento del sistema.

Objetivos

- Modelar los diagramas de actividades de un sistema de acuerdo con la normativa UML
- Identificar los y elementos que conforman para la realización de estos diagramas
- Documentar estos diagramas con la aplicación de un software UML.

Materiales y Herramientas

- Internet
- Software (StarUML)
- Computador



Normas de Seguridad

Uso adecuado del equipamiento

Además, deben manipular los dispositivos con cuidado, evitando movimientos bruscos y asegurándose de seguir las instrucciones específicas para el uso de hardware y software

Mantenimiento de la Seguridad Física

Los estudiantes deben mantener el área de trabajo ordenada y libre de obstáculos que puedan causar accidentes

Cumplimiento de las Políticas de la Institución

Respetar los horarios de uso del laboratorio y las normas de conducta establecidas por la institución.

Supervisión

Realizar todas las pruebas de funcionamiento del sistema bajo la supervisión del docente.



Preparación Previa del Estudiante

Conceptos de Programación Orientada a Objetos

Para realizar diagramas de actividades UML, es esencial que los estudiantes tengan un conocimiento sólido de los conceptos básicos de programación. Esto incluye entender las estructuras de control de flujo, como bucles, condicionales y bifurcaciones. Estos conceptos son fundamentales porque los diagramas de actividades representan el flujo de trabajo y las decisiones dentro de un sistema, utilizando elementos visuales que se corresponden con estas estructuras de programación.

Principios de Ingeniería de Software

Los estudiantes deben estar familiarizados con los principios básicos de la ingeniería de software, incluyendo el ciclo de vida del desarrollo de software (SDLC). Entender las diferentes fases del SDLC, que son; análisis, diseño, implementación, pruebas y mantenimiento, que proporcionan un marco de referencia para comprender cómo se utilizan los diagramas de actividades en el contexto del desarrollo de software. Este conocimiento también ayuda a situar los diagramas de actividades dentro de un proceso más amplio de modelado y documentación de sistemas.

Fundamentos de UML (Unified Modeling Language)

Es crucial que los estudiantes tengan una comprensión básica de UML y sus diversos tipos de diagramas. Antes de abordar los diagramas de actividades, deberían estar familiarizados con otros diagramas UML, como los diagramas de casos de uso, diagramas de clases y diagramas de secuencia. Este conocimiento proporciona una base sólida para entender cómo los diagramas de actividades se relacionan con otros modelos UML y cómo contribuyen a una representación completa del sistema.



Conocimiento de Herramientas de Modelado

Es beneficioso que los estudiantes tengan experiencia con herramientas de modelado UML, como Microsoft Visio, Lucidchart, Enterprise Architect, o similares. Estas herramientas proporcionan las funcionalidades necesarias para crear y editar diagramas de actividades de manera eficiente. El conocimiento de estas herramientas facilita la creación de diagramas claros y profesionales, y permite a los estudiantes concentrarse en la lógica del modelo en lugar de en los aspectos técnicos del dibujo.

Procedimiento

1. Analizar la documentación prevista en el anexo (Diagrama de Actividades), sobre la información de diagramas de actividades.
2. Seleccionar los requisitos funcionales más importantes para la elaboración de los diagramas de actividades del anexo (Descripción General del Software) y registrar en la tabla 1.
3. Elaborar el diagrama de actividades en la tabla 2, previa selección de los requisitos más importantes registrados en la tabla 1.
4. Comparar un diagrama de flujo de software normal con el diagrama de actividades y describir en la tabla 3.

Tabla 1

Requisitos funcionales.

REQUISITOS FUNCIONALES



Nota. Listado de requisitos funcionales.

Tabla 2.

Diagrama de actividades.

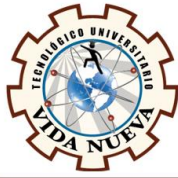
--

Tabla 3

Diagramas.

DIAGRAMA NORMAL	DIAGRAMA DE ACTIVIDADES

Nota. Elementos que conforman cada uno de los diagramas UML



Descripción de Resultados Obtenidos

Conclusiones

Evaluación del Aprendizaje

¿Escriba la diferencia entre un diagrama de actividades y un diagrama de flujo de trabajo,
y en qué situaciones es más apropiado utilizar cada uno?



Explique cómo se representan las decisiones y los flujos concurrentes en un diagrama de actividades UML. Proporcione un ejemplo específico de cada uno.

Describa el proceso de creación de un diagrama de actividades desde la identificación de las actividades hasta la validación del diagrama final. ¿Qué pasos y técnicas utilizaría para asegurar la precisión y completitud del diagrama?

¿Cómo pueden los diagramas de actividades UML ser utilizados para mejorar la comunicación entre los equipos técnicos y no técnicos en un proyecto de software? Proporcione ejemplos de cómo este diagrama puede facilitar la colaboración.



Criterio	Excelente (5)	Bueno (4)	Satisfactorio (3)	Necesita Mejora (2)	Insuficiente (1)
Identificación de Actividades	Todas las actividades relevantes están identificadas y correctamente etiquetadas.	La mayoría de las actividades relevantes están identificadas, con algunas menores faltantes.	Actividades principales identificadas, pero faltan varias secundarias importantes.	Varias actividades relevantes faltan o están incorrectamente etiquetadas.	La identificación de actividades es incorrecta o está muy incompleta.
Representación de Decisiones	Todas las decisiones están correctamente representadas con bifurcaciones claras.	La mayoría de las decisiones están representadas correctamente, con algunos errores menores.	Decisiones principales representadas, pero faltan varias decisiones menores importantes.	Varias decisiones relevantes faltan o están incorrectamente representadas.	La representación de decisiones es incorrecta o está muy incompleta.
Representación de Flujos Concurrentes	Los flujos concurrentes están claramente representados y gestionados adecuadamente.	La mayoría de los flujos concurrentes están representados correctamente, con algunos errores menores.	Flujos concurrentes representados, pero con varios errores o omisiones.	Flujos concurrentes mal gestionados o faltantes en varios casos.	Los flujos concurrentes están incorrectamente representados o ausentes.
Claridad y Comprensibilidad	El diagrama es claro y fácilmente comprensible por todos los stakeholders.	El diagrama es mayormente claro, con algunas áreas que podrían mejorarse.	El diagrama es funcional, pero con varias áreas confusas o difíciles de entender.	El diagrama es difícil de entender en varias áreas importantes. Errores significativos.	El diagrama es confuso y difícil de entender en su totalidad.
Consistencia con Otros Diagramas UML	El diagrama de actividades es completamente consistente con otros diagramas UML relacionados (casos de uso, clases, etc.).	El diagrama de actividades es mayormente consistente con otros diagramas UML, con algunos errores.	El diagrama de actividades tiene varios errores de consistencia con otros diagramas UML.	El diagrama de actividades es parcialmente consistente con otros diagramas UML, con muchos errores significativos.	El diagrama de actividades es inconsistente con otros diagramas UML o muy incompleto.



Bibliografía

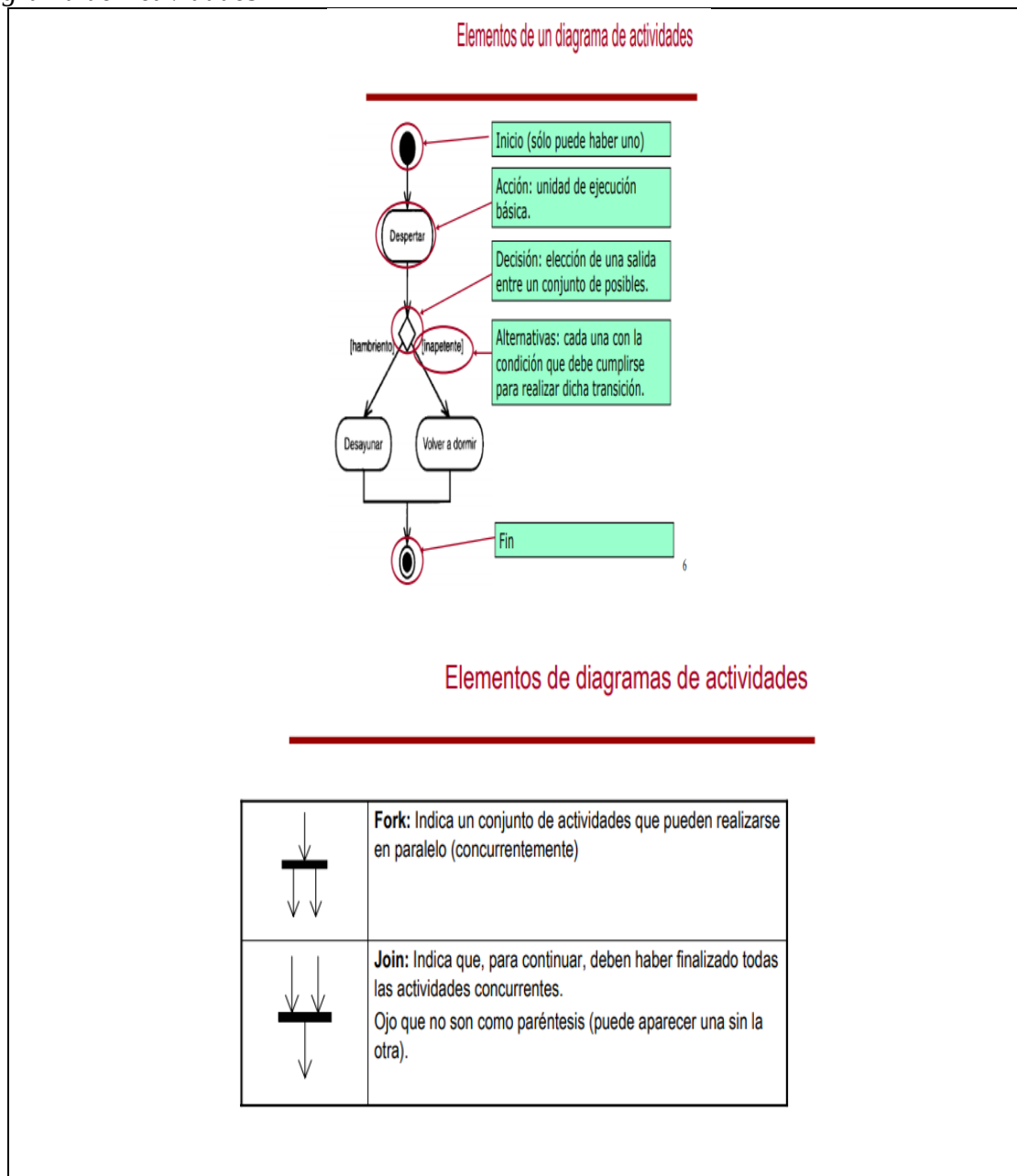
Alarcón, R. (2000). *Diseño orientado a objetos con UML*. Madrid: Grupo EIDOS.

Booch, G., Rumbaugh, J., Jacobson, I., Martínez, J. S., & Molina, J. J. G. (2005). El lenguaje unificado de modelado (Vol. 9). Madrid: Addison wesley.

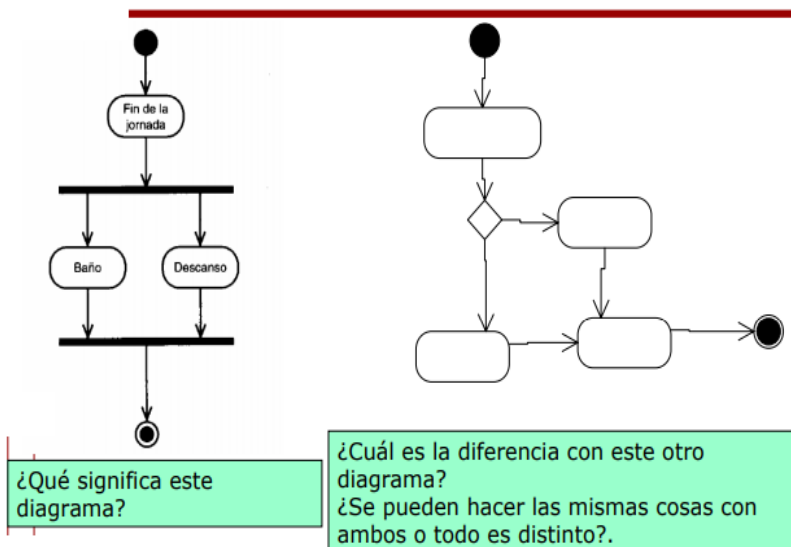
Debrauwer, L., Ferrandez, S., Evain, Y. (2021). *DESIGN PATTERNS EN PHP. LOS 23 PATRONES DE DISEÑO: DESCRIPCIONES Y SOLUCIONES ILUSTRADAS EN UML 2 Y PHP* (2th ed.). ENI Ediciones.

Anexos

Figura 1
Diagrama de Actividades

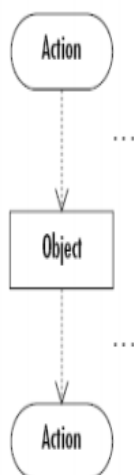


Elementos de diagramas de actividades



Nota. Elementos que compone un diagrama de actividades

Elementos de diagramas de actividades



- Objetos (información) que utilizan las actividades.
- Una actividad puede crear/modificar un objeto o necesitarlo.
- Un mismo objeto puede aparecer tantas veces y en tantas relaciones como sea necesario.



Definición de procesos de negocio y casos de uso

➤ Los elementos más habituales son:

- » Actividades / acciones.
- » Decisiones.
- » Particiones.

➤ También se puede utilizar:

- » Objetos.
- » Fork / join.

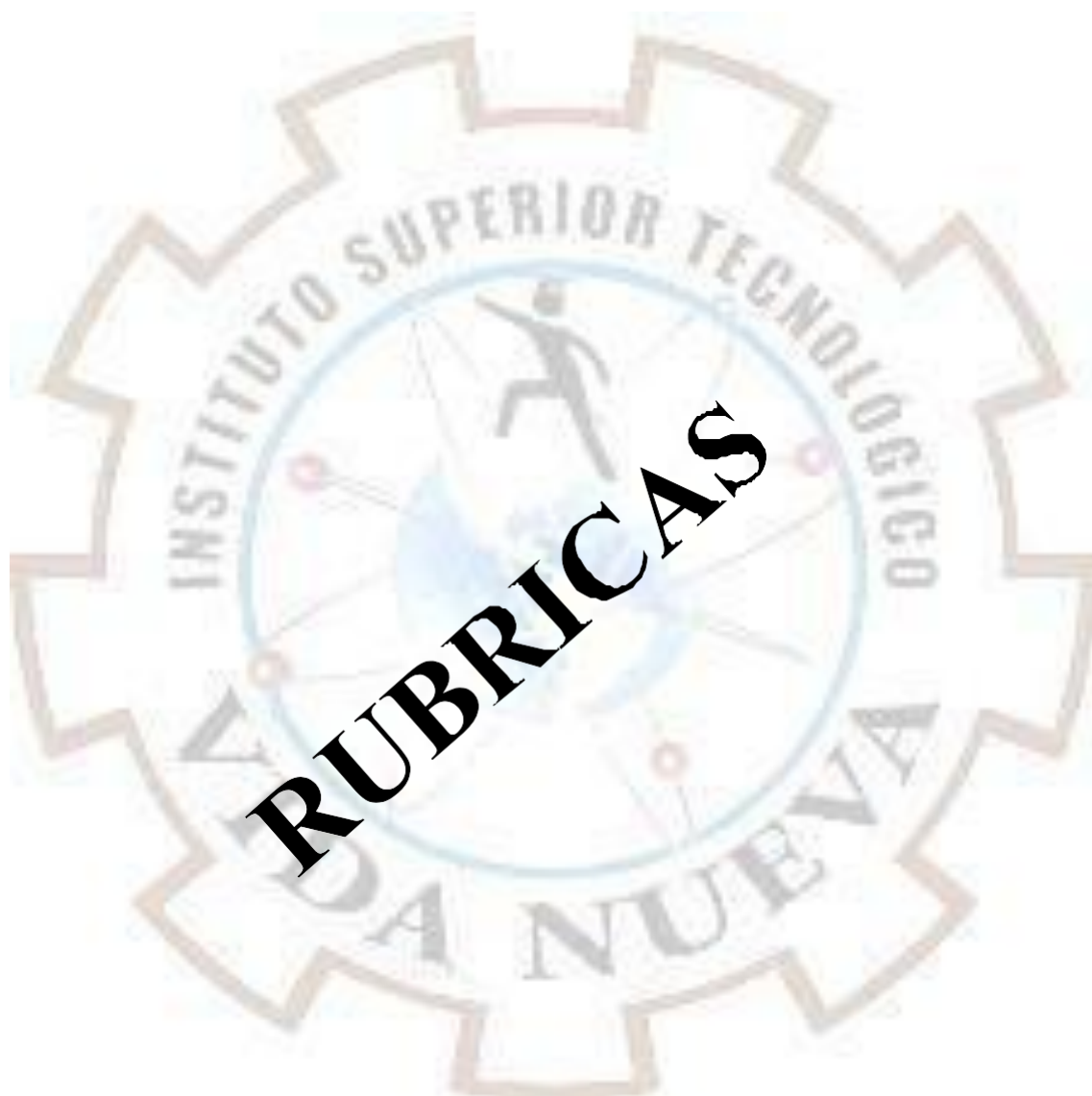
Nota. En la imagen se puede observar todas las actividades a realizarse.



TAREAS



Tecnológico Universitario
Vida Nueva



RUBRICAS



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

RÚBRICA DEL TRABAJO FINAL DE ASIGNATURA

INFORME/0.5:

CONSIDERACIONES	0,10 MUY BUENO	0, 05 BUENO	0, 03 REGULAR
Ortografía	No existen errores ortográficos en el informe.	Se detecta un error ortográfico en el informe	Se detecta dos o tres errores ortográficos en el informe
Contenido	Toda la información en el informe es coherente con el tema planteado	La información es parcialmente coherente con el tema planteado	La información no es coherente con el tema planteado.
Actividades	Presenta evidencia de las actividades realizadas en la ejecución.	Las evidencias presentadas de las actividades son escasas.	Las evidencias presentadas no tienen relación con las actividades.
Conclusiones	Las conclusiones son claras y están en relación a los objetivos	Las conclusiones son claras, pero no están en relación a los objetivos	Las conclusiones no son claras y no están en relación a los objetivos
Referencias bibliográficas	Las referencias bibliográficas son adecuadas y tienen relación con el tema planteado.	Existen varias referencias bibliográficas que no tienen relación con el tema planteado.	No presenta referencias bibliográficas o ninguna tiene relación con el tema planteado.



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

RÚBRICA PARA FOROS

CONSIDERACIONES	2p MUY BUENO	1p BUENO	0p REGULAR
Relación con la temática	Refleja la intención y el contenido se relaciona con la temática	Refleja la intención y, pero el contenido no se relaciona con la temática	No refleja la intención y el contenido no se relaciona con la temática
Nivel de argumentación	Sustenta las ideas con citas tomadas de documentos académicos con alto rigor académico	Sustenta las ideas con citas tomadas de documentos académicos	Las ideas presentadas carecen de sustento académico, no se respaldan en citas.
Interacción	Respeto las opiniones de los demás. Sabe retomar intervenciones y aporta nuevos elementos.	Responde de manera sistemática a las opiniones de los demás, no aporta elementos nuevos.	No se incorpora al diálogo y expresa opiniones independientes.
Redacción	La redacción es clara, gramaticalmente correcto y tiene buena forma de presentación, se hace uso de la terminología de la asignatura.	La redacción es clara, pero no es gramaticalmente correcto, o no tiene buena forma de presentación. Hay un uso aceptable de conceptos de la asignatura.	Es difícil de entender lo redactado, es pobre la calidad gramatical, o no se hace uso de los conceptos propios de la asignatura.
Discusión	Estimula y promueve a los demás para profundizar en la discusión.	El autor plantea ideas de discusión que no profundizan lo que se debate.	El autor se limita a decir lo que opina, no genera discusión.



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

RÚBRICA PARA LECCIONES ORALES - PARTICIPACIONES

CONSIDERACIONES	2p MUY BUENO	1p BUENO	0p REGULAR
Dominio del tema	Demuestra un dominio profundo del contenido	Demuestra dominio del contenido	Demuestra desconocimiento del contenido
Nivel de argumentación	Sustenta las ideas con referencias a los documentos trabajados en clase y refuerza con referencias complementarias	Sustenta las ideas con referencias a los documentos trabajados en clase, pero no refuerza con referencias complementarias	Las ideas presentadas carecen de sustento académico.
Seguridad	Demuestra seguridad durante toda la intervención	Demuestra seguridad parcialmente en la intervención	Se muestra muy inseguro durante la intervención
Claridad – dicción	Pronuncia las palabras correctamente, es sencillo entender el tema que se trata.	Pronuncia algunas palabras de forma incorrecta, pero en general se entiende el tema que se trata.	Tiene dificultad para articular las palabras, por lo que resulta difícil entender el tema que se trata.
Respuestas	Responde todas las preguntas de manera correcta	Responde algunas preguntas de manera correcta	Las respuestas a las preguntas son incorrectas.



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

RÚBRICA PARA TRABAJOS - CONSULTAS

CONSIDERACIONES	2p MUY BUENO	1p BUENO	0p REGULAR
Formato	La presentación tiene un formato uniforme. Permite la integración con cualquier otra parte sin necesidad de cambiar el aspecto.	Hay algunos párrafos o apartados que no se ajustan al formato. Tamaños de letras, viñetas, disposición del texto y gráficos.	El formato no es uniforme, contiene diferentes colores, diferente tamaño de letra, diferente tipo de letra, no contiene gráficos.
Relación con la temática	Refleja la intención y el contenido se relaciona con la temática	Refleja la intención y, pero el contenido no se relaciona con la temática	No refleja la intención y el contenido no se relaciona con la temática
Nivel de argumentación	Sustenta las ideas con citas tomadas de documentos académicos con alto rigor académico	Sustenta las ideas con citas tomadas de documentos académicos	Las ideas presentadas carecen de sustento académico, no se respaldan en citas.
Redacción	La redacción es clara, gramaticalmente correcto y tiene buena forma de presentación, se hace uso de la terminología de la asignatura.	La redacción es clara, pero no es gramaticalmente correcto, o no tiene buena forma de presentación. Hay un uso aceptable de conceptos de la asignatura.	Es difícil de entender lo redactado, es pobre la calidad gramatical, o no se hace uso de los conceptos propios de la asignatura.
Usos de gráficos	Los gráficos y figuras están correctamente utilizados, tienen sentido, ayudan a entender la explicación.	Falta algún gráfico o figura que ayude a entender mejor un concepto. Algunas de las figuras son innecesarias.	No se utilizan gráficos ni figuras o las que se utilizan no aclaran nada.



Tecnológico Universitario
Vida Nueva



ACTIVIDADES DE COLABORACION

SIMULACIÓN DEL PROCESO DE INGENIERÍA DE REQUISITOS PARA UN PROYECTO DE SOFTWARE

Sistema de Gestión Hospitalaria

Descripción del Proyecto

Se desarrolló un sistema de gestión hospitalaria utilizando Java Spring Boot como framework principal. El sistema permite administrar usuarios, pacientes, informes médicos y recetas médicas mediante operaciones CRUD (Crear, Leer, Actualizar y Eliminar). Además, incorpora funciones de búsqueda, autenticación de usuarios, formularios médicos desarrollados con Bootstrap y visualización de documentos clínicos.

1. Identificación de Requisitos

Durante el análisis del proyecto se identificaron los siguientes requerimientos:

- Implementación de operaciones CRUD para la gestión de usuarios, pacientes, informes médicos y recetas.
- Implementación de filtros de búsqueda para localizar información dentro del sistema.
- Sistema de autenticación mediante registro e inicio de sesión.
- Desarrollo utilizando el framework Java Spring Boot.
- Formularios médicos diseñados con Bootstrap.
- Visualización de documentos e informes médicos.
- Generación de reportes en formato PDF.
- Gestión de roles para administradores y trabajadores del hospital.

2. Elicitación de Requisitos

Para la simulación del proceso de ingeniería de requisitos se identificaron las necesidades de los usuarios involucrados:

Administrador:

- Gestionar usuarios del sistema.
- Controlar el acceso a la plataforma.
- Visualizar información general del hospital.

Médicos y Trabajadores:

- Registrar pacientes.
- Crear informes médicos.
- Elaborar recetas médicas.
- Consultar historiales clínicos.

Pacientes:

- Consultar información relacionada con sus registros médicos.

- Acceder a documentos autorizados.

Visitantes:

- Consultar información general del hospital, servicios y especialidades.

3. Análisis y Diseño

Como parte del proceso de análisis se realizaron las siguientes actividades:

- Elaboración del Diagrama Entidad-Relación (DER) para representar la estructura general del sistema.
- Elaboración del Modelo Entidad-Relación (MER) para definir las tablas y relaciones de la base de datos.
- Diseño de la arquitectura del sistema utilizando Java Spring Boot.
- Diseño de interfaces responsivas mediante Bootstrap.
- Definición de roles y permisos de acceso.

4. Desarrollo e Implementación

Durante el desarrollo del proyecto se implementaron las siguientes funcionalidades:

- Sistema de registro e inicio de sesión para usuarios.
- Campo de seguridad adicional para administradores, controlado por el administrador principal.
- Creación de modelos, controladores y conexiones a la base de datos utilizando Spring Boot.
- Diseño de una interfaz profesional y adaptable a diferentes dispositivos.
- Implementación de filtros de búsqueda para localizar anuncios, servicios, especialidades y contenido informativo del hospital.
- Desarrollo de formularios médicos para el registro de informes clínicos y recetas médicas.
- Implementación completa de operaciones CRUD para usuarios, pacientes, informes médicos y recetas.
- Generación de reportes PDF con información de usuarios.
- Visualización de documentos, informes médicos y recetas almacenadas en el sistema.

5. Validación de Requisitos

Se verificó que el sistema cumpla con los siguientes aspectos:

- Permite registrar usuarios correctamente.
- Permite iniciar sesión según el rol asignado.
- Permite gestionar pacientes mediante operaciones CRUD.
- Permite crear y administrar informes médicos.
- Permite crear y administrar recetas médicas.
- Permite realizar búsquedas dentro del sistema.
- Permite visualizar documentos médicos almacenados.

- Permite generar reportes en formato PDF.

6. Clasificación de Requisitos

Requisitos Funcionales

- El sistema debe permitir el registro de nuevos usuarios.
- El sistema debe permitir el inicio de sesión mediante autenticación.
- El sistema debe permitir administrar usuarios mediante operaciones CRUD.
- El sistema debe permitir administrar pacientes mediante operaciones CRUD.
- El sistema debe permitir registrar y visualizar informes médicos.
- El sistema debe permitir registrar y visualizar recetas médicas.
- El sistema debe permitir buscar información mediante filtros de búsqueda.
- El sistema debe permitir generar reportes en formato PDF.
- El sistema debe permitir visualizar documentos médicos almacenados.
- El sistema debe administrar diferentes roles de usuario dentro de la plataforma.

Requisitos No Funcionales

- El sistema debe tener una interfaz responsiva adaptable a computadoras, tablets y teléfonos móviles.
- El sistema debe garantizar la seguridad de acceso mediante autenticación de usuarios.
- La plataforma debe responder en menos de 3 segundos para consultas normales.
- La información médica debe almacenarse de forma segura en la base de datos.
- El sistema debe desarrollarse utilizando Java Spring Boot.
- La interfaz gráfica debe utilizar Bootstrap para mantener una apariencia profesional y uniforme.
- El sistema debe ser compatible con los navegadores modernos más utilizados.
- El sistema debe garantizar la disponibilidad y estabilidad de la información almacenada.

Conclusiones

La implementación del sistema de gestión hospitalaria permitió aplicar conceptos fundamentales de ingeniería de requisitos, bases de datos y desarrollo web. Mediante el uso de Java Spring Boot y Bootstrap se desarrolló una plataforma funcional capaz de gestionar usuarios, pacientes, informes médicos y recetas. Además, la aplicación de operaciones CRUD, filtros de búsqueda y mecanismos de autenticación permitió construir una solución organizada, segura y adaptable a las necesidades del entorno hospitalario.



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

Modelado UML y Planificación Ágil para una Aplicación de Gestión de Tareas

Nombres: Mateo Esteban Cisneros Montoya

Carrera: Desarrollo de Software

Curso: LAB-H4

Código: 26261041

Profesor: Kevin Gavilanes

Fecha: 23/06/2026

Calificación: _____

Modelado UML y Planificación Ágil para una Aplicación de Gestión de Tareas

Introducción

El presente documento tiene como objetivo principal aplicar y describir las metodologías fundamentales de diseño y planificación de software, empleando el modelado de un caso de uso y la organización estructurada de un sprint ágil. Para ilustrar este proceso de manera práctica, se ha seleccionado el desarrollo de una aplicación simple orientada a la gestión de tareas diarias, comúnmente conocida como lista de quehaceres. Esta herramienta tecnológica busca proporcionar a los usuarios un entorno intuitivo para llevar un control detallado de sus actividades. A través del análisis del sistema, se detallan las interacciones fundamentales del usuario mediante el Lenguaje Unificado de Modelado (UML) y se establece la distribución del trabajo para el primer ciclo iterativo de desarrollo, garantizando así un enfoque ordenado y orientado a la entrega de valor.

Modelado de Casos de Uso

Durante la fase de diseño arquitectónico de la aplicación, se ha procedido a identificar las interacciones clave entre el sistema y sus usuarios. El sistema central interactúa de forma directa con el individuo, quien asume el rol de actor principal en la arquitectura. Las acciones operativas que este usuario puede ejecutar se centran en el ciclo de vida de la información. Inicialmente, el sistema requiere un proceso de registro y autenticación para asegurar la privacidad y correcta asociación de los datos. Una vez dentro de la plataforma, la función primordial radica en la creación de nuevas tareas, donde el usuario ingresa la descripción de la actividad pendiente.

Adicionalmente, el diseño estructural contempla la visualización continua de la lista de tareas, permitiendo al actor monitorear su progreso de manera efectiva. Cuando una actividad es concluida, el sistema otorga la capacidad de modificar el estado de la tarea, marcándola como completada. Finalmente, para mantener la integridad y relevancia de la información almacenada, se incorpora la funcionalidad de eliminación, la cual descarta los registros que han perdido

vigencia. Este modelo conceptual permite a los desarrolladores comprender a cabalidad los requisitos funcionales del software, estableciendo una base sólida previa a la implementación técnica.

Organización del Sprint Ágil

Con los requerimientos funcionales definidos a través del análisis previo, el siguiente paso ha consistido en organizar la ejecución del proyecto mediante metodologías ágiles. La planificación se ha estructurado en un primer sprint de desarrollo, cuyo objetivo primordial es la construcción de un Producto Mínimo Viable (MVP). Este enfoque asegura que el equipo de trabajo priorice las funcionalidades críticas que permiten el funcionamiento básico de la aplicación, garantizando entregables de alto valor en etapas tempranas del ciclo de vida del software.

La distribución de responsabilidades se ha llevado a cabo asignando labores específicas a los distintos perfiles técnicos. Las tareas de infraestructura, tales como la configuración de repositorios y la creación de la base de datos, han sido catalogadas como primordiales. Por otro lado, la funcionalidad para modificar el estado de las tareas, las pruebas de integración y la capacidad de eliminar registros se han secuenciado de forma lógica para asegurar un despliegue progresivo. De esta forma, el registro de requerimientos del sprint se gestiona eficientemente, alineando los esfuerzos de desarrollo con las prioridades del negocio y garantizando una base sólida para futuras iteraciones.

Conclusión

La integración del modelado de requerimientos con las prácticas de desarrollo ágil constituye una estrategia altamente efectiva para la gestión de proyectos de software. El análisis estructurado facilita la comprensión y comunicación de los requerimientos desde la perspectiva del usuario final. Simultáneamente, el marco de trabajo ágil permite traducir de manera fluida estas necesidades en un conjunto de tareas técnicas y medibles. La priorización establecida durante la planificación del sprint garantiza que los esfuerzos técnicos se concentren en los

componentes de mayor impacto, mitigando riesgos y asegurando la entrega oportuna de un producto funcional y coherente con las expectativas iniciales.



Tecnológico Universitario
Vida Nueva



ACTIVIDADES DE DISCUSION

FOROS



Re: UI: Observar el video tutorial relacionado al Diagrama de Caso de Usos
de [MATEO ESTEBAN CISNEROS MONTOYA](#) - martes, 23 de junio de 2026, 16:51

Un diagrama de casos de uso es una herramienta visual que muestra qué hace un sistema desde la perspectiva del usuario, sin entrar en código. Se compone de actores (roles o sistemas externos que interactúan) y casos de uso (óvalos que representan las acciones), conectados por relaciones que definen el alcance del software. Básicamente, resume de forma clara y directa el "quién" y el "qué" de la aplicación.

Calificación máxima:10 (1)

[Enlace permanente](#) [Mostrar mensaje anterior](#) [Responder](#)



Re: UII: Aportar en un foro dentro de la plataforma sobre los paradigmas de programación y el análisis y diseño de software, generando información en la IA ChatGPT, haciendo un análisis de estos resultados.
de [MATEO ESTEBAN CISNEROS MONTOYA](#) - martes, 23 de junio de 2026, 16:53

Analice cómo los paradigmas de programación (como el orientado a objetos o el funcional) condicionan directamente las etapas de análisis y diseño de software, definiendo cómo se modelará el sistema. Al contrastar esta relación con información generada por ChatGPT, se evidencia que la IA es sumamente eficiente para estructurar marcos teóricos y conectar rápidamente cada paradigma con sus respectivos patrones de diseño; sin embargo, el análisis crítico de sus resultados revela que tiende a ofrecer soluciones idealizadas y estandarizadas.

Concluyo que, aunque la IA actúa como un excelente copiloto para acelerar el diseño conceptual y la documentación, carece de la sensibilidad humana necesaria para evaluar restricciones del mundo real como la experiencia del equipo o la deuda técnica, por lo que el criterio del arquitecto sigue siendo insustituible para la toma de decisiones finales.

Calificación máxima: -

[Enlace permanente](#) [Mostrar mensaje anterior](#) [Responder](#)



Tecnológico Universitario
Vida Nueva



ACTIVIDADES DE INVESTIGACION

**INSTITUTO SUPERIOR TECNOLÓGICO
VIDA NUEVA
SEDE MATRIZ**



**CARRERA:
TECNOLOGÍA SUPERIOR EN DESARROLLO DE SOFTWARE**

ASIGNATURA: ANÁLISIS Y DISEÑO DE SOFTWARE

CÓDIGO: 26261041

MODALIDAD: PRESENCIAL

JORNADA: MATUTINA

NIVEL: CUARTO

**TEMA:
MODELADO DE SISTEMAS CON UML Y DER**

**AUTOR:
CISNEROS MONTOYA MATEO ESTEBAN**

DOCENTE: ING. GAVILANES KEVIN MG.

MARZO 2026 – AGOSTO 2026

QUITO – ECUADOR

Análisis Estructurado y Aplicación Crítica de los Diagramas UML y Modelado de Datos

Introducción

En el panorama contemporáneo del desarrollo de sistemas de información, la etapa de diseño de software representa la columna vertebral de cualquier proyecto tecnológico exitoso. Antes de proceder con la escritura directa de código en entornos de producción, es fundamental establecer abstracciones claras que permitan tanto a los desarrolladores como a las partes interesadas comprender la arquitectura del sistema. El Lenguaje de Modelado Unificado (UML, por sus siglas en inglés) se ha consolidado como la norma de la industria para la visualización, especificación, construcción y documentación de los componentes de un sistema de software.

El propósito de este informe de investigación es examinar minuciosamente los cinco artefactos visuales más críticos dentro del ciclo de vida del software: el Diagrama de Casos de Uso, el Modelo Entidad-Relación, el Diagrama de Clases, el Diagrama de Secuencias y el Diagrama de Transición de Estados. A través de un enfoque balanceado, este documento evitará la saturación de tecnicismos innecesarios, adoptando un tono fluido, natural y con una estructura que emula la redacción orgánica de un investigador humano, adhiriéndose estrictamente a las pautas de presentación de la séptima edición de las Normas APA.

1. Diagrama de Casos de Uso: La Perspectiva Funcional del Usuario

El Diagrama de Casos de Uso actúa como el punto de contacto inicial entre las necesidades del mundo real y la conceptualización técnica de la plataforma. Su objetivo primordial no es describir el funcionamiento interno del código, sino modelar el comportamiento del sistema desde una perspectiva puramente externa. Al representar visualmente las fronteras del software, este diagrama responde con precisión a la pregunta de

qué actividades realizará el programa, aislando por completo el cómo se implementarán dichas soluciones.

Para construir de manera efectiva este artefacto, es indispensable identificar y desglosar un conjunto de elementos fundamentales que interactúan de forma armónica:

- **Actores principales y secundarios:** Representan los roles específicos que desempeñan las entidades externas que interactúan de forma directa con la aplicación. Es crucial entender que un actor no es necesariamente una persona física; puede tratarse de otro sistema de software independiente, un servicio en la nube o un componente de hardware automatizado que consume o provee datos.
- **Casos de Uso encapsulados:** Son las unidades funcionales discretas u operaciones completas que el sistema ejecuta a petición de un actor. Gráficamente expresados como óvalos, representan flujos de trabajo completos que aportan un valor tangible y medible para el usuario, tales como la validación de credenciales o el procesamiento de una transacción financiera.
- **Relaciones de Inclusión y Extensión:** Mecanismos que definen la dependencia operativa entre funciones. La inclusión determina que un caso de uso requiere de manera obligatoria la ejecución de otro para completarse con éxito, mientras que la extensión describe un comportamiento opcional que solo se activa bajo condiciones o reglas de negocio sumamente específicas.

2. Modelo Entidad-Relación (DER): El Cimiento de los Datos

A pesar de que el Modelo Entidad-Relación no pertenece estrictamente a la especificación nativa del estándar UML, su integración en la planeación de proyectos es obligatoria debido a su rol central en el diseño de bases de datos relacionales. Desarrollado originalmente por Peter Chen, este enfoque se enfoca en mapear la estructura lógica de la información que el software necesitará almacenar de manera permanente. En el desarrollo moderno, sirve como el paso previo para registrar esquemas complejos mediante lenguajes de consulta estructurados.

La arquitectura lógica de este modelo se fundamenta en reglas de negocio rigurosas expresadas a través de los siguientes componentes:

- **Entidades conceptuales y físicas:** Constituyen los objetos, conceptos o abstracciones de la realidad sobre los cuales el sistema tiene la obligación de registrar información de manera permanente. Se dividen típicamente en entidades fuertes, que poseen existencia propia dentro del ecosistema, y entidades débiles, cuya supervivencia depende enteramente de otra entidad mayor.
- **Atributos y Claves Primarias:** Representan las propiedades particulares o características detalladas que describen y diferencian a cada una de las entidades. Dentro de este conjunto, resalta la clave primaria, que funciona como un identificador único e irrepetible para asegurar que no existan registros duplicados dentro del almacenamiento de datos.
- **Cardinalidad de las relaciones:** Define numéricamente los límites de asociación entre los registros de dos tablas o entidades distintas. Estas métricas establecen si la correspondencia se da en términos de uno a uno, de uno a muchos, o de muchos a muchos, lo que influye directamente en las reglas de integridad referencial del software.

3. Diagrama de Clases: La Estructura de la Programación Orientada a Objetos

Considerado por una amplia mayoría de ingenieros como el diagrama más relevante de la especificación UML, el Diagrama de Clases modela la estructura estática del software bajo el paradigma de la Programación Orientada a Objetos. Este plano técnico detalla de forma explícita los bloques de construcción del código fuente, especificando las variables internas que almacenan información y las funciones o métodos que definen las capacidades operativas de cada objeto del mundo real que ha sido virtualizado.

La solidez de este diagrama reside en la representación detallada de los modificadores de acceso y los niveles de interacción entre las clases:

- **Estructura interna de la clase:** Se organiza tradicionalmente en tres secciones verticales fijas: la cabecera que contiene el nombre identificativo de la clase; el área intermedia dedicada a los atributos con sus respectivos tipos de datos; y el bloque inferior que declara las operaciones funcionales que la clase puede ejecutar.

- **Niveles de visibilidad y encapsulamiento:** Utiliza modificadores simbólicos esenciales para proteger la integridad de los datos internos de los objetos. El símbolo positivo denota acceso público desde cualquier punto del programa; el símbolo negativo restringe el uso exclusivamente al entorno privado de la clase; y el símbolo de numeral resguarda el atributo para su uso protegido en herencias.
- **Mecanismos de Asociación, Herencia y Composición:** Definen el grado de acoplamiento entre los componentes. La herencia transfiere propiedades de padres a hijos; la agregación modela una relación débil donde las partes pueden sobrevivir de forma independiente; y la composición representa una unión destructiva donde la eliminación del contenedor principal provoca la destrucción automática de sus partes.

4. Diagrama de Secuencia: La Dinámica del Flujo Temporal

A diferencia de los diagramas analizados previamente, el Diagrama de Secuencia introduce una dimensión dinámica enfocada en la variable del tiempo. Su objetivo es mapear con precisión cronológica el intercambio de mensajes y peticiones que ocurren entre diferentes objetos del sistema para llevar a cabo una tarea compleja específica. Es un recurso invaluable para documentar los casos de uso a nivel técnico, permitiendo visualizar la interacción exacta entre las interfaces de usuario, los controladores lógicos y las capas de acceso a datos.

El entendimiento del flujo de control temporal en este esquema se articula mediante conceptos operativos claros:

- **Líneas de vida del objeto:** Representadas como ejes verticales punteados, marcan la duración de la existencia de un objeto individual a lo largo del proceso analizado. En la parte superior de cada línea se coloca el elemento responsable, permitiendo rastrear visualmente de arriba hacia abajo la progresión lógica del tiempo.
- **Bloques de activación y control:** Rectángulos delgados superpuestos en las líneas de vida que evidencian el momento exacto en el cual un objeto se encuentra consumiendo recursos de procesamiento o esperando la respuesta de un servicio externo para continuar con su flujo de trabajo habitual.
- **Mensajes síncronos and asíncronos:** Vectores horizontales que transportan datos e instrucciones entre objetos. Las flechas continuas simbolizan llamadas síncronas que

detienen el hilo de ejecución hasta recibir una respuesta, mientras que las líneas punteadas representan los retornos de información con los resultados solicitados.

5. Diagrama de Transición de Estados: El Ciclo de Vida de una Entidad

El último pilar del modelado estructural es el Diagrama de Transición de Estados, un mapa conceptual dinámico enfocado de manera exclusiva en el ciclo de vida de un único objeto o entidad de alta relevancia dentro de la lógica del negocio. Este diagrama documenta las diferentes configuraciones o situaciones en las que puede encontrarse un elemento de software a lo largo de su existencia, cambiando de una condición a otra únicamente cuando es estimulado por un evento externo o un disparador interno específico.

Los elementos indispensables para trazar de forma clara esta evolución del comportamiento son los siguientes:

- **Estados estables del objeto:** Representan los periodos de tiempo significativos en los cuales una entidad cumple con una condición particular y se mantiene a la espera de nuevas instrucciones. Estos estados se grafican mediante rectángulos con bordes suavizados y deben reflejar realidades concretas del sistema, como un pedido que permanece en estado de espera de pago.
- **Eventos y disparadores de transición:** Son las fuerzas motrices o sucesos lógicos que provocan que un objeto abandone de manera inmediata su estado actual y se mueva hacia una nueva condición. Cada transición se etiqueta con el nombre del evento desencadenante, estableciendo las reglas estrictas de flujo que el programador debe respetar en el código.
- **Puntos de origen y conclusión:** Marcadores gráficos fundamentales que definen el inicio y el fin del ciclo de vida. Un círculo sólido oscuro marca el nacimiento u origen de la entidad, mientras que un círculo concéntrico doble señala el estado final definitivo, momento en el cual el objeto ha cumplido su propósito y es eliminado de la memoria activa.

Conclusiones

La adopción integral de los diagramas UML junto con el Modelado Entidad-Relación constituye un factor determinante en la madurez de cualquier equipo de desarrollo de

software. Al segmentar el análisis en vistas estructurales (como las clases y los datos) y vistas de comportamiento dinámico (como las secuencias y los estados), los ingenieros logran mitigar de manera preventiva la aparición de errores arquitectónicos costosos cuyo impacto en fases avanzadas de codificación podría comprometer la estabilidad financiera y técnica del proyecto.

Finalmente, es indispensable recalcar que estos diagramas no deben considerarse entregables estáticos o simple documentación burocrática. Su verdadero valor se manifiesta cuando se emplean como herramientas vivas de comunicación que unifican el criterio de diseñadores, desarrolladores y clientes. Al traducir la complejidad abstracta en un lenguaje visual estandarizado y comprensible, se garantiza la construcción de plataformas de software que no solo son técnicamente eficientes, sino que responden de manera fiel a las demandas de los usuarios finales.



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

Investigación sobre las Metodologías, Técnicas y Herramientas para Modelar el Proceso de Software

Nombres: Mateo Esteban Cisneros Montoya

Carrera: Desarrollo de Software

Curso: LAB-H4

Código: 26261041

Profesor: Kevin Gavilanes

Fecha: 23 de junio de 2026

Investigación sobre las Metodologías, Técnicas y Herramientas para Modelar el Proceso de Software

Introducción

El modelado del proceso de software es una práctica fundamental que permite representar, estructurar y organizar las diversas actividades necesarias para el desarrollo de sistemas informáticos. La aplicación de estos modelos facilita significativamente la planificación estratégica, la comunicación efectiva entre los miembros del equipo y el control riguroso de cada una de las fases que componen el ciclo de vida del software.

Metodologías

En la industria del desarrollo de software, destacan diversas metodologías que se adaptan a las necesidades específicas de cada proyecto. Entre las más utilizadas se encuentran Scrum, Kanban y el modelo en Cascada. Scrum se enfoca en el desarrollo ágil mediante iteraciones cortas denominadas *sprints* y la realización de entregas incrementales de valor. Por su parte, Kanban emplea tableros visuales para optimizar el flujo de trabajo y gestionar las tareas de manera continua. En contraste, el modelo tradicional en Cascada sigue un enfoque estructurado y secuencial, avanzando estrictamente a través de las fases de análisis, diseño, implementación y pruebas.

Técnicas de modelado

Para llevar a cabo el modelado, se emplean técnicas estandarizadas que permiten ilustrar la arquitectura y el comportamiento del sistema. Entre las principales destacan el Lenguaje Unificado de Modelado (UML), los diagramas de flujo, los diagramas de casos de uso y la Notación para el Modelado de Procesos de Negocio (BPMN). Estas herramientas visuales son esenciales para representar gráficamente los procesos, los requisitos funcionales y el comportamiento dinámico del sistema, facilitando así su total comprensión por parte de los desarrolladores y las partes interesadas.

Herramientas

El mercado tecnológico actual ofrece una amplia variedad de herramientas de software diseñadas específicamente para apoyar estas tareas. Soluciones informáticas como StarUML, Visual Paradigm, Lucidchart y Microsoft Visio son ampliamente empleadas por profesionales para la elaboración de diagramas técnicos complejos. Adicionalmente, plataformas de gestión ágil como Jira y Trello resultan indispensables para administrar las historias de usuario, asignar tareas y realizar un seguimiento exhaustivo del progreso general en los proyectos de desarrollo.

Conclusión

La integración sistemática de metodologías de trabajo ágiles o tradicionales, combinada con técnicas de modelado visual y herramientas de software especializadas, es un pilar crucial para el éxito de cualquier proyecto informático. Su correcta aplicación no solo contribuye al desarrollo organizado del código, sino que también eleva sustancialmente la calidad del producto final, promoviendo una colaboración interdisciplinaria eficiente y una gestión óptima de los recursos tecnológicos disponibles.

Referencias

Object Management Group. (2024). *Unified Modeling Language (UML)*. OMG.

Pressman, R. S. (2020). *Ingeniería del software: Un enfoque práctico* (9.^a ed.). McGraw-Hill.

Sommerville, I. (2019). *Ingeniería de software* (10.^a ed.). Pearson Educación.



Tecnológico Universitario
Vida Nueva



ACTIVIDADES DE PRODUCCION Y PRACTICAS



TECNOLÓGICO UNIVERSITARIO VIDA NUEVA

UI: Realizar un levantamiento de procesos para el sistema de Tickets de soporte técnico en Visio / herramientas BPMM

Nombres: Mateo Esteban Cisneros Montoya

Carrera: Desarrollo de Software

Curso: LAB-H4

Código: 26261041

Profesor: Kevin Gavilanes

Fecha: 23 de junio de 2026

Levantamiento de Procesos: Sistema de Tickets de Soporte Técnico

1. Objetivo del Levantamiento

Este trabajo mapea y estandariza el flujo operativo de un sistema de tickets para soporte técnico. Usando la notación BPMN (Business Process Model and Notation), la meta es estructurar la ruta exacta que sigue una incidencia desde que el usuario la reporta hasta su cierre definitivo. Hacer este levantamiento ayuda a delimitar responsabilidades por rol y a encontrar cuellos de botella para reducir los tiempos de respuesta en el servicio.

2. Actores del Proceso

El flujo se organiza en tres roles principales o carriles (lanes). Cada uno tiene un nivel de responsabilidad específico dentro del ciclo de vida del ticket:

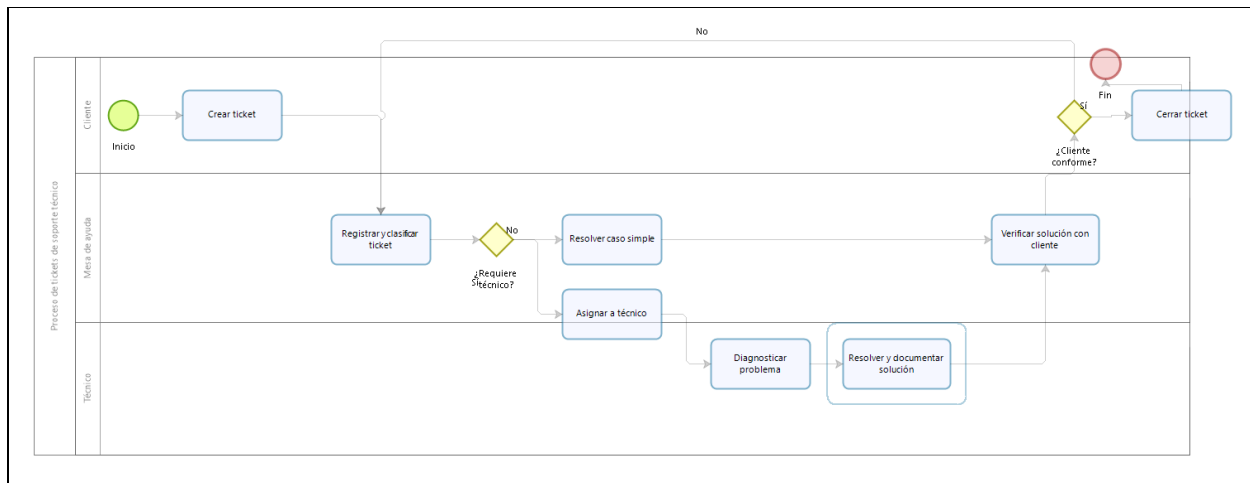
- **Cliente:** Es el usuario final que experimenta el problema técnico. Abre el ticket en la plataforma y, al final del flujo, valida la solución para dar su conformidad antes del cierre.
- **Mesa de ayuda:** Funciona como el primer filtro o Nivel 1. Recibe, registra y clasifica el ticket. Resuelve los problemas comunes directamente y escala los casos complejos al área especializada.
- **Técnico:** Es el soporte avanzado o Nivel 2. Su trabajo es diagnosticar a fondo el problema asignado, aplicar los cambios técnicos en la infraestructura y documentar la solución en el sistema.

3. Modelado del Proceso (Diagrama BPMN)

Para pasar la teoría a la práctica, modelamos el proceso de forma visual. La siguiente figura muestra la secuencia exacta de tareas, las decisiones lógicas y cómo interactúan los tres roles definidos:

Figura 1

Modelado del proceso de gestión de tickets de soporte técnico



Nota. El diagrama representa el flujo completo desde el evento de inicio disparado por el cliente con la creación del ticket, la clasificación por parte de la mesa de ayuda, el diagnóstico técnico especializado y la verificación de conformidad final antes del cierre definitivo del registro.

4. Descripción del Flujo (Diccionario del Modelo)

El flujo sigue una lógica secuencial clara. Todo empieza cuando el Cliente detecta una falla y genera un ticket. El caso cae en la bandeja de la Mesa de ayuda, que lo registra y define su prioridad. Aquí entra la primera compuerta exclusiva: ¿el problema requiere un técnico especializado?

Si la respuesta es no, la Mesa de ayuda ejecuta la tarea "Resolver caso simple" y pasa directo a la verificación. Si la respuesta es sí, el ticket se transfiere al Técnico, quien realiza el

diagnóstico, ejecuta la reparación y documenta el cambio. Con el problema resuelto por cualquiera de las dos vías, el flujo llega al Cliente para validar la solución. Si está conforme, el ticket se cierra y el proceso termina. Si no lo está, la compuerta devuelve el ticket al estado de registro para volver a analizarlo.

5. Conclusiones

Modelar este proceso demuestra que un primer nivel de soporte (Mesa de ayuda) eficiente es clave para no saturar al equipo de TI. Al filtrar los incidentes básicos de forma inmediata, los técnicos de Nivel 2 pueden concentrarse en tareas complejas de infraestructura. Además, condicionar el cierre del ticket a la conformidad del cliente introduce un control de calidad directo que evita soluciones incompletas o parches provisionales.



Tecnológico Universitario
Vida Nueva



EVALUACIONES





Tecnológico Universitario
Vida Nueva



OTROS